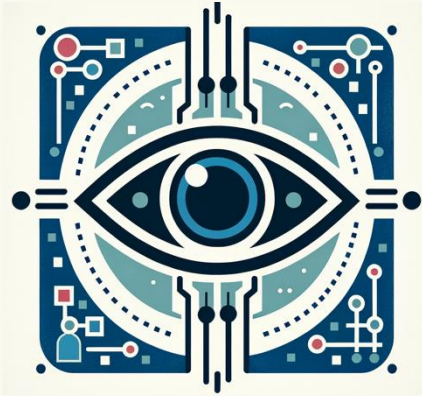




COMPUTER VISION

Corners, Blobs, Local Features

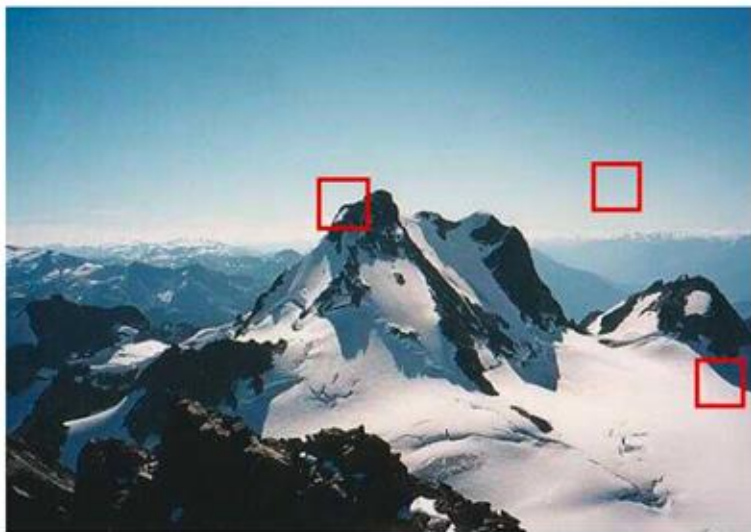
What kinds of features might one use to establish a set of correspondences between these images?



Properties:

- Semantically meaningful?
 - Unfeasible → it would require interpretation of the scene content
- Underlying intensity patterns
 - Unique in a given image, but that can be easily be found in another image

Source: Szeliski



Difficult to
localise



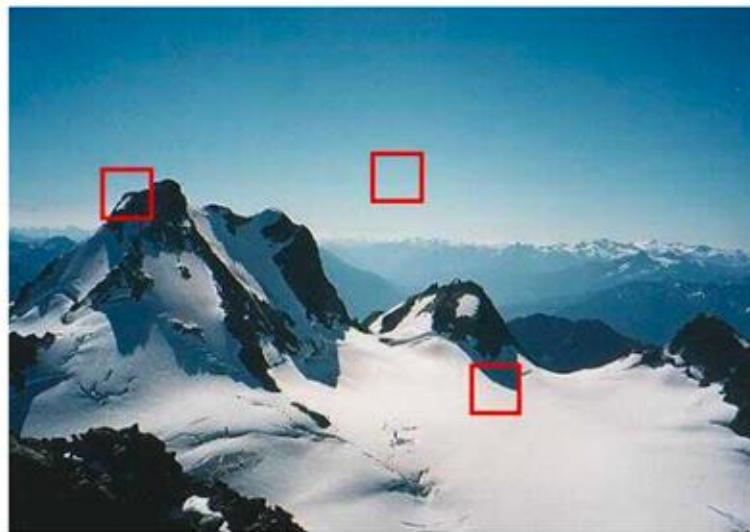
Better, but
many
matches



Easier to
localise

(edge)

(corner)



Difficult to
localise



Better, but
many
matches



Easier to
localise

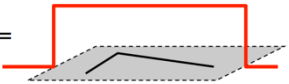
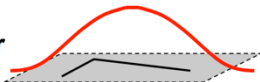
How can we find unique patches?

- We want a patch that is unique in the image
 - Calculate distance between patch and **every other patch** → lot of computation
- Instead, we could think about self-difference:
 - How well does image match **shifted version of itself**?

$$E(u, v) = \sum_{x, y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

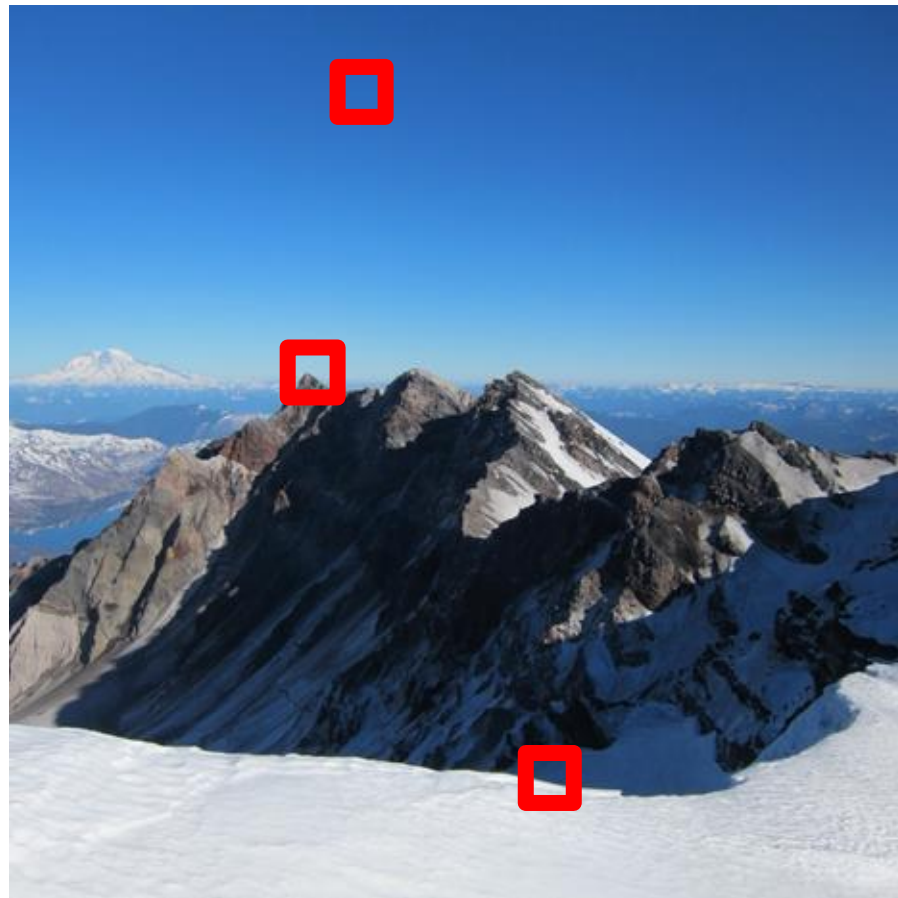
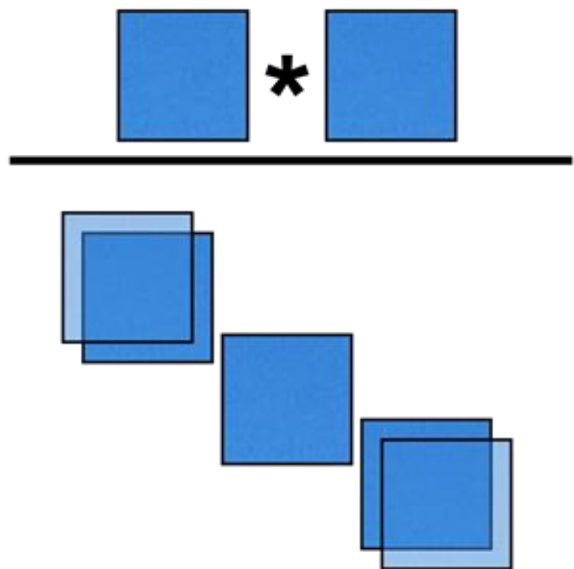
Diagram illustrating the components of the equation:

- Window function** ($w(x, y)$)
- Shifted intensity** ($I(x + u, y + v)$)
- Intensity** ($I(x, y)$)

Window function $w(x, y) =$  **1 in window, 0 outside** or  **Gaussian**

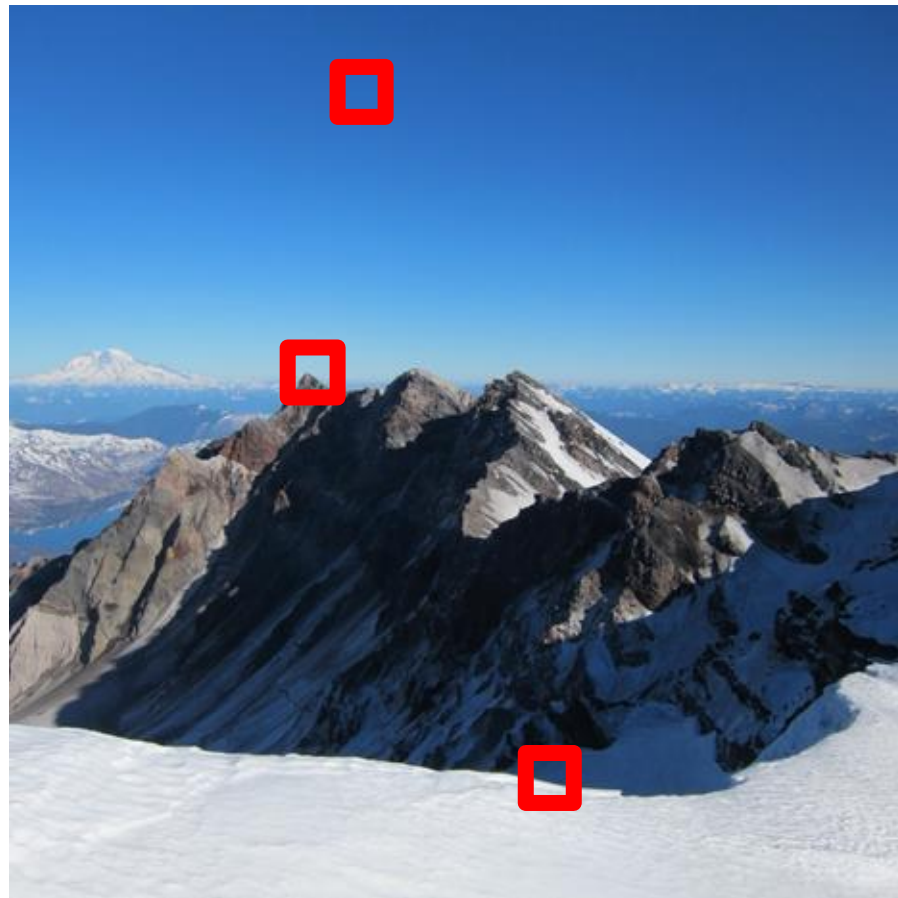
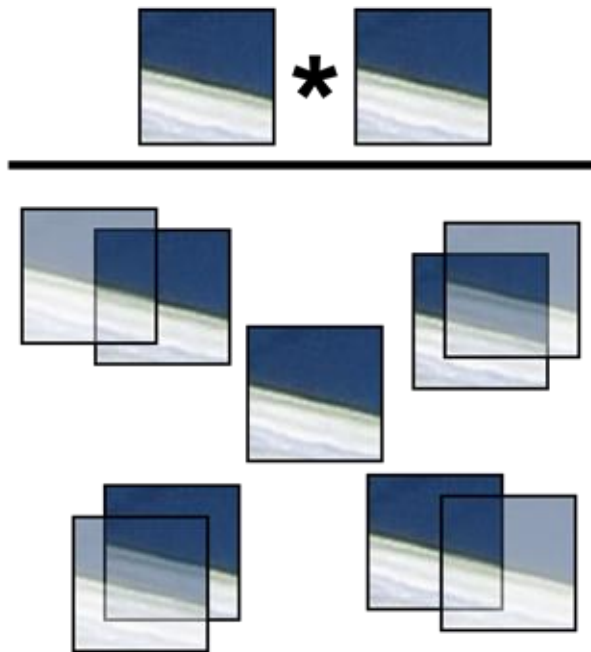
Self-difference

Sky: low everywhere



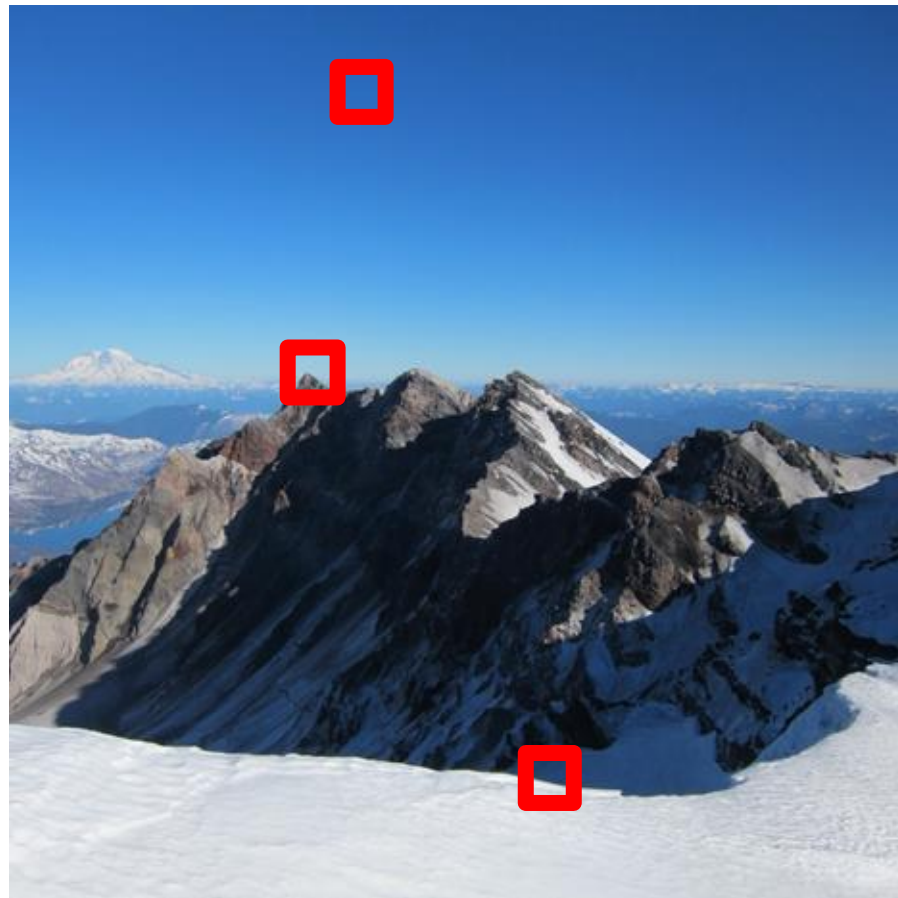
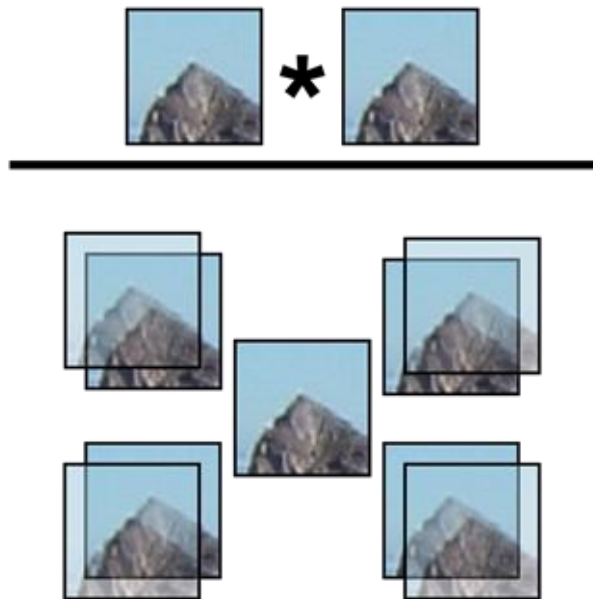
Self-difference

Edge: low along edge

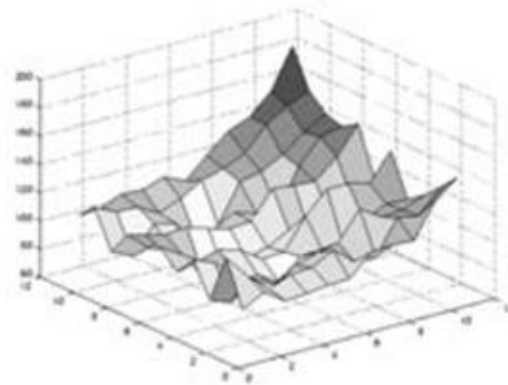
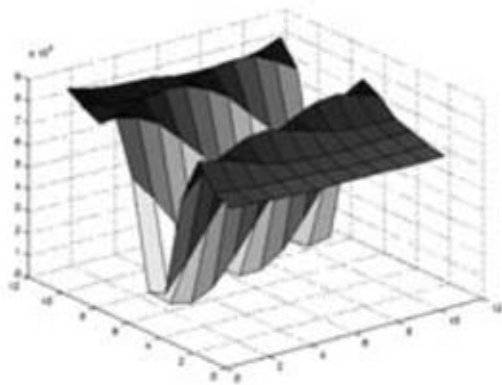
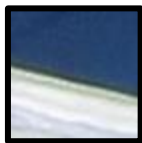
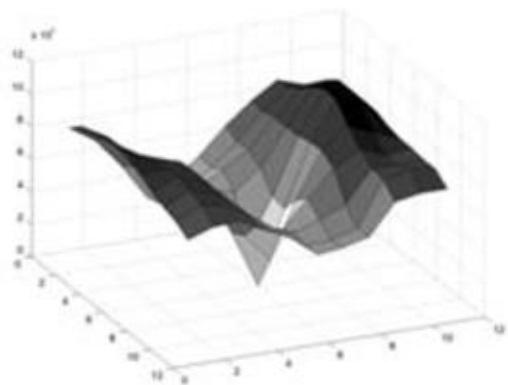


Self-difference

Corner: mostly high



Self-difference



Self-difference

Change in appearance of window $w(x,y)$ for shift $[u,v]$:

$$E(u, v) = \sum_{x,y} w(x, y) [I(x + u, y + v) - I(x, y)]^2$$

We want to discover how E behaves for small shifts

But this is very slow to compute naively.

$O(\text{window_width}^2 * \text{shift_range}^2 * \text{image_width}^2)$

$O(11^2 * 11^2 * 600^2) = 5.2 \text{ billion}$

14.6 thousand per pixel in your image

Approximate self-difference

- Look at nearby gradients I_x and I_y
- If gradients are mostly zero, not a lot going on
 - Low self-difference
- If gradients are mostly in one direction, edge
 - Still low self-difference
- If gradients are strong in two directions, corner
 - High self-difference, good patch!

Harris Detector Formulation

The measure of change can be approximated by:
(Taylor series expansion)

$$E(u, v) \approx \begin{bmatrix} u & v \end{bmatrix} M \begin{bmatrix} u \\ v \end{bmatrix}$$

where M is a 2×2 matrix computed from image derivatives:

$$M = \sum_{x,y} w(x,y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

↑
Sum over image region – area we are checking for corner

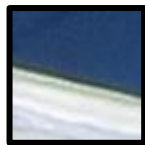
weighted sum of gradients near pixel
(depending on the window function)

← Gradient with respect to x, times gradient with respect to y

$$M = \begin{bmatrix} \sum I_x I_x & \sum I_x I_y \\ \sum I_x I_y & \sum I_y I_y \end{bmatrix} = \sum \begin{bmatrix} I_x \\ I_y \end{bmatrix} \begin{bmatrix} I_x & I_y \end{bmatrix}$$

What does this matrix reveal?

- Eigenvectors/values of this matrix summarise the gradients information
 - direction $\rightarrow$ eigenvectors
 - magnitude $\rightarrow$ eigenvalues
- **Corner response function** based on the eigenvalues:



$$\lambda_1 \gg \lambda_2$$
$$\lambda_2 \gg \lambda_1$$



$$\lambda_1 \text{ and } \lambda_2 \text{ are large,}$$
$$\lambda_1 \sim \lambda_2;$$



$$\lambda_1 \text{ and } \lambda_2 \text{ are}$$
$$\text{small;}$$

$$C = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2$$

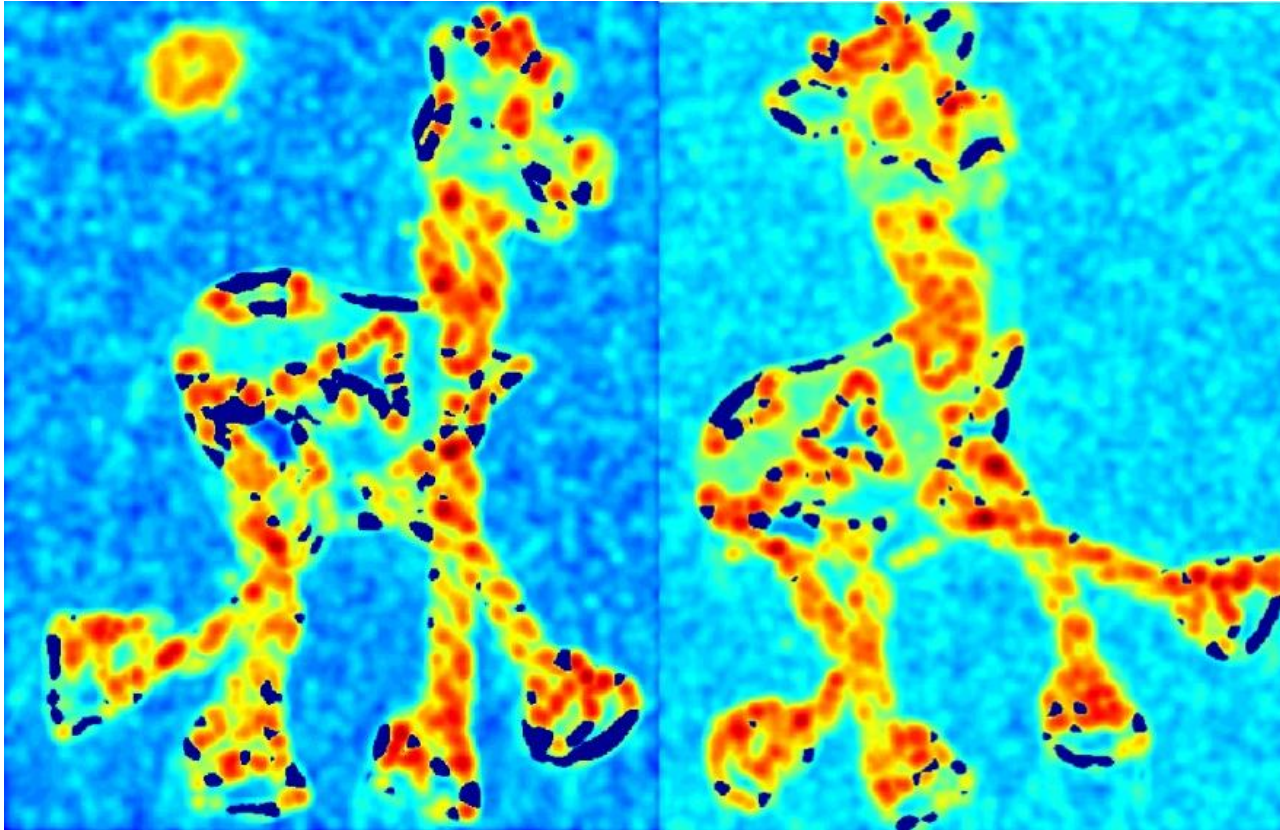
α : constant (0.04 to 0.06)

Harris corner detector



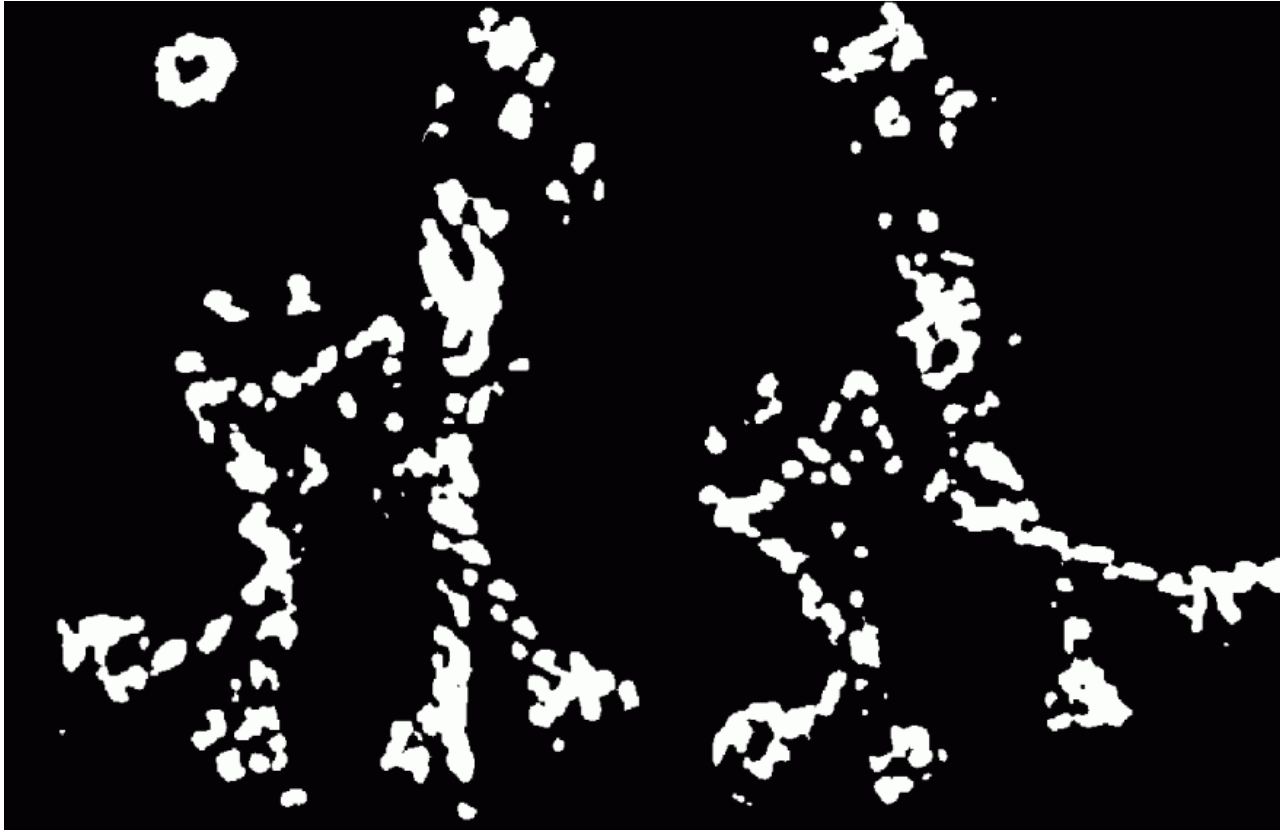
Harris corner detector

Compute corner response C



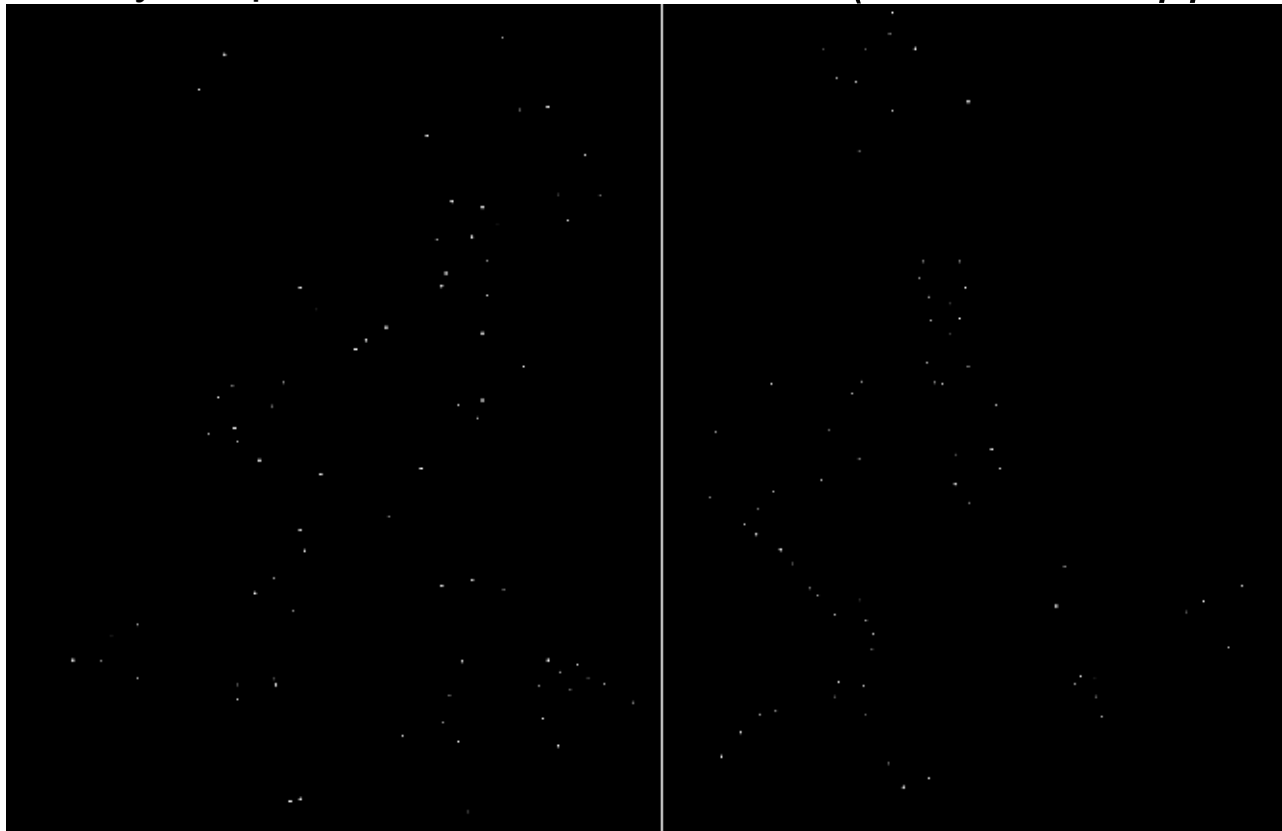
Harris corner detector

Find points with large corner response: $C > \text{threshold}$

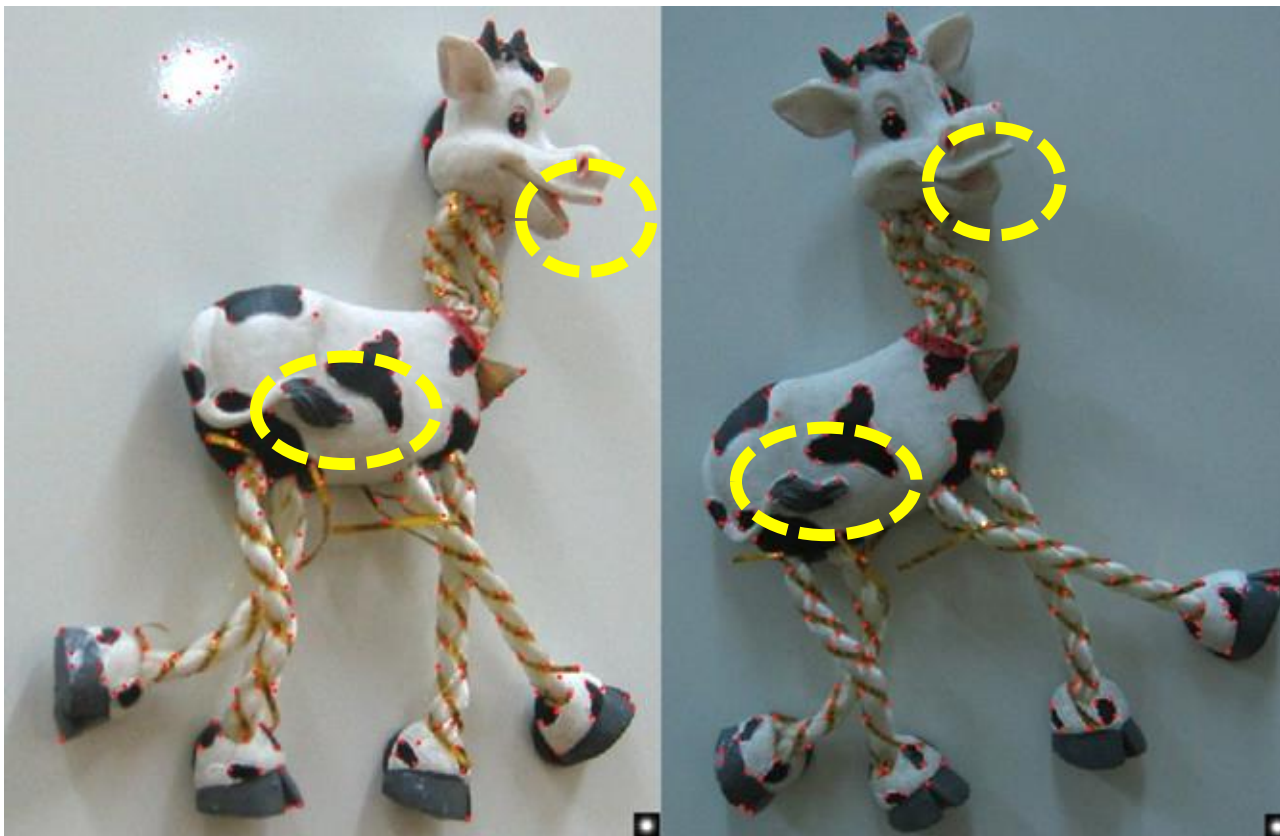


Harris corner detector

Take only the points of local maxima of C (*non-max suppression*)



Harris corner detector



Harris corner detector



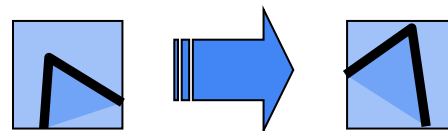


Scale invariance

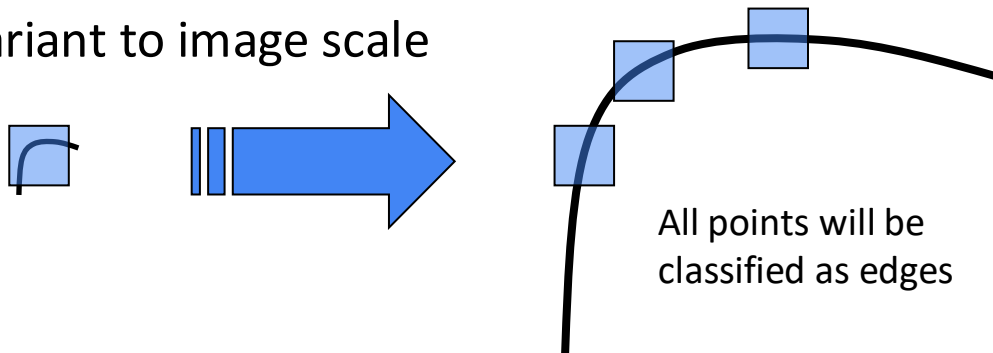
- Properties of the Harris corner detector

- Rotation invariance

rotates but its shape (i.e. eigenvalues) remains the same



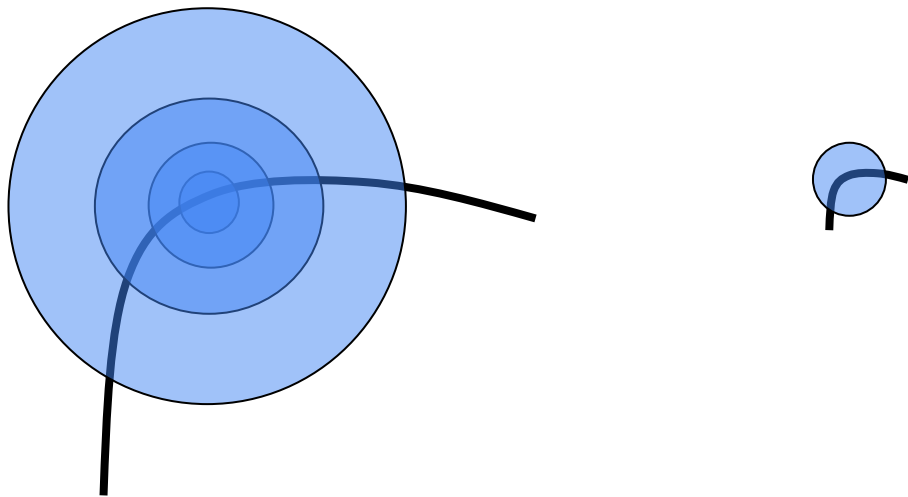
- Not invariant to image scale



How can we detect **scale invariant** interest points?

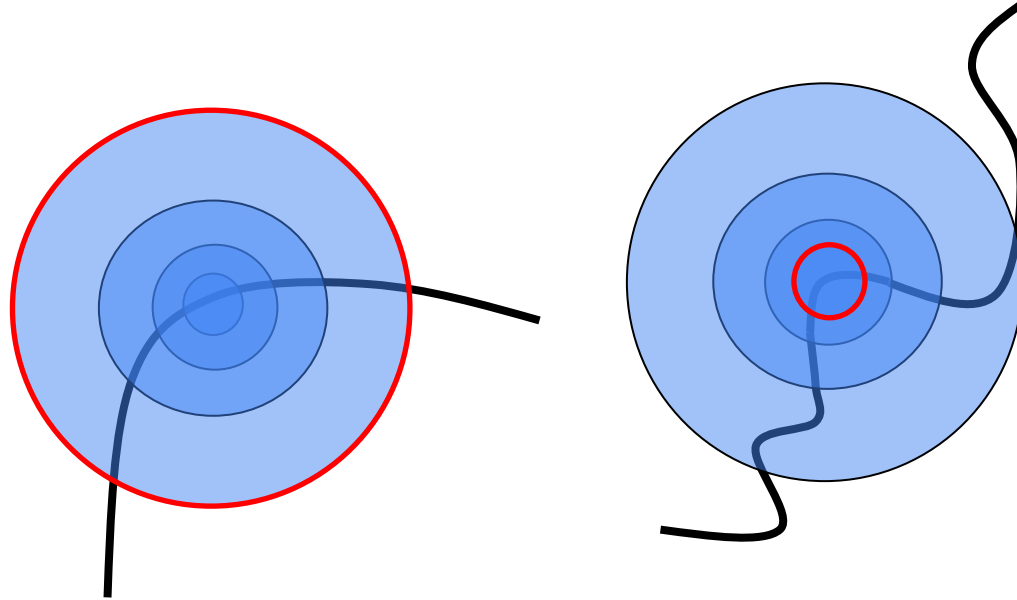
Scale invariant detection

- Consider regions (e.g. circles) of different sizes around a point
- Find regions of corresponding sizes that will look the same in both images?



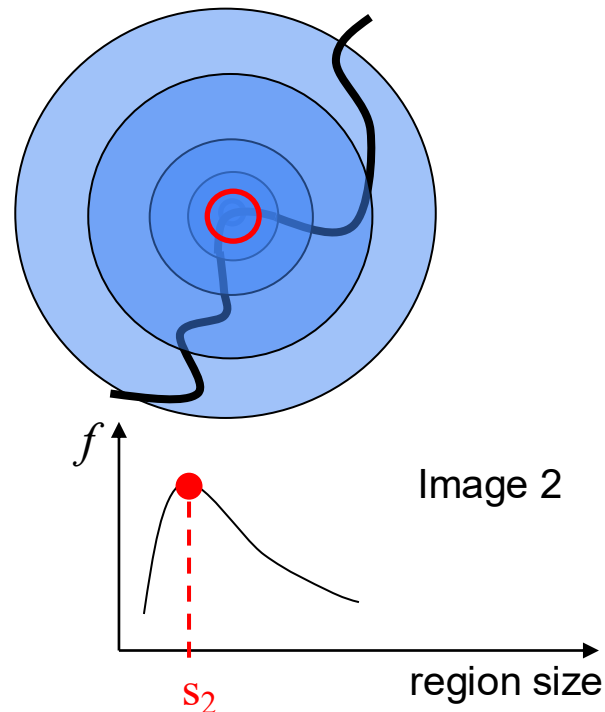
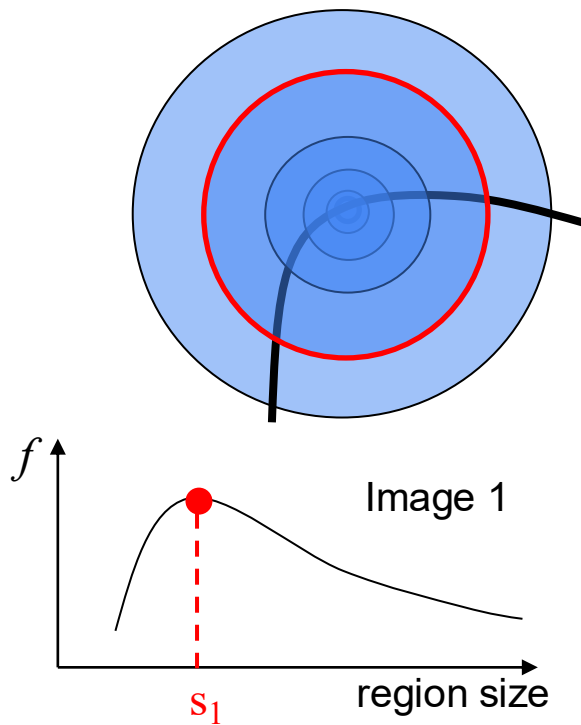
Scale invariant detection

- The problem: how do we choose corresponding circles *independently* in each image?



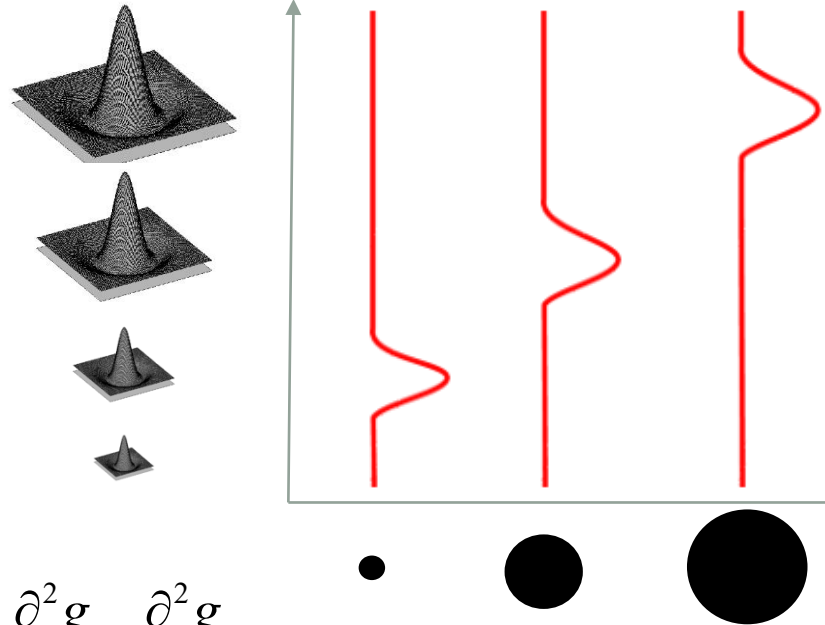
Scale invariant detection

Find scale that gives local maxima of some response function f in both position and scale



Scale invariant detection

Laplacian-of-Gaussian = “blob” detector



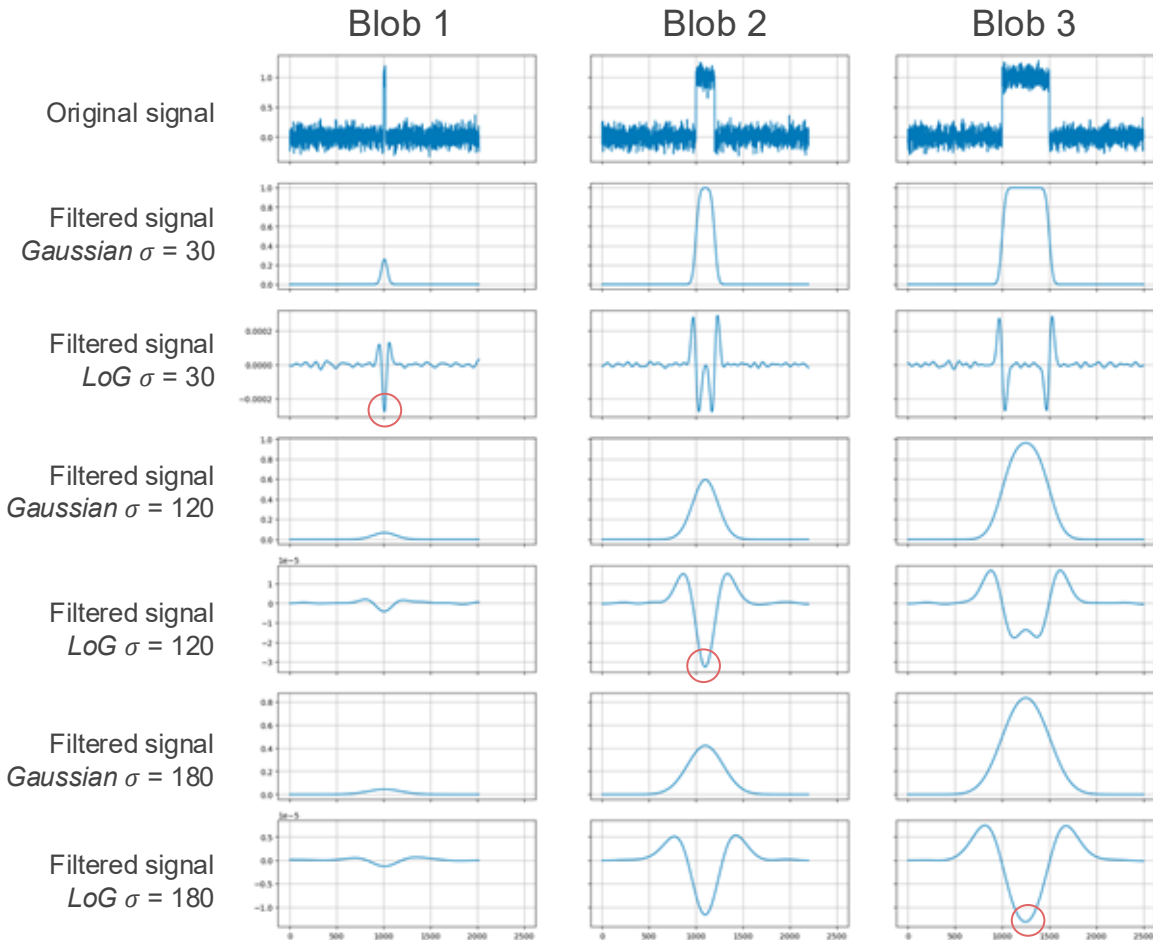
$$\nabla^2 g = \frac{\partial^2 g}{\partial x^2} + \frac{\partial^2 g}{\partial y^2}$$

1D signals

σ controls the space-scale of the smoothness

The response of the operator will depend on σ to find blobs at different scales.

So, we need to explore different σ s



Increase σ value



Detection of larger blobs

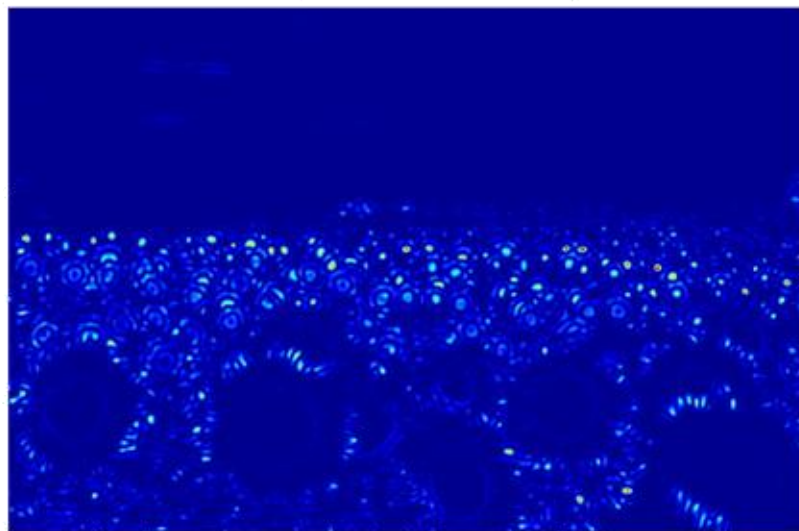
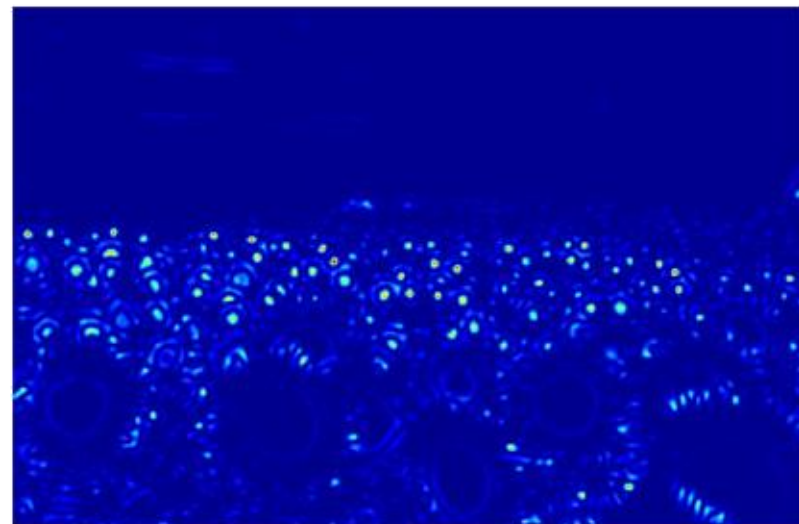
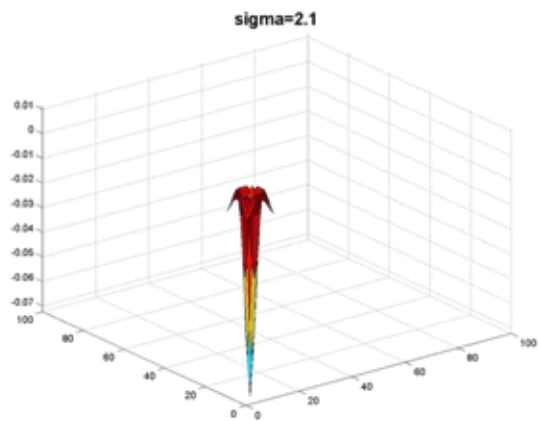
Images

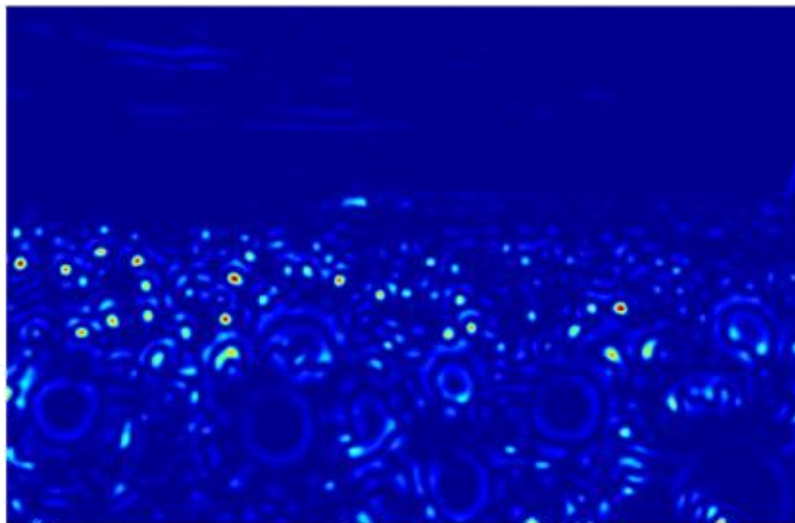
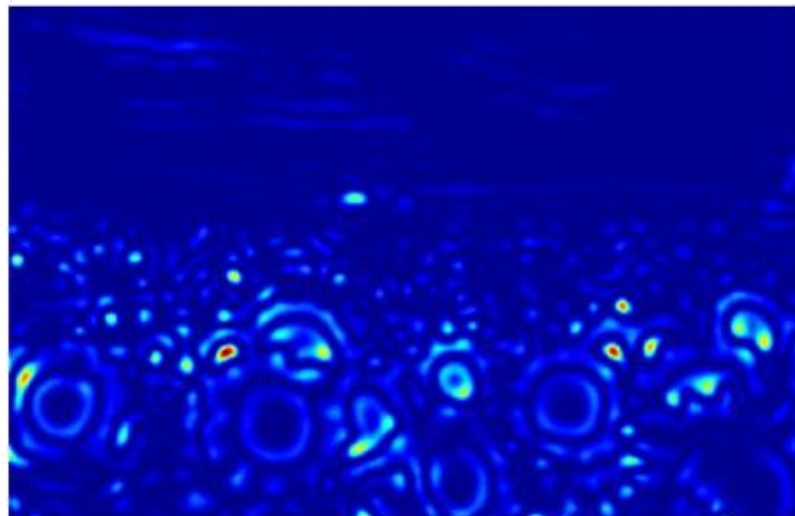
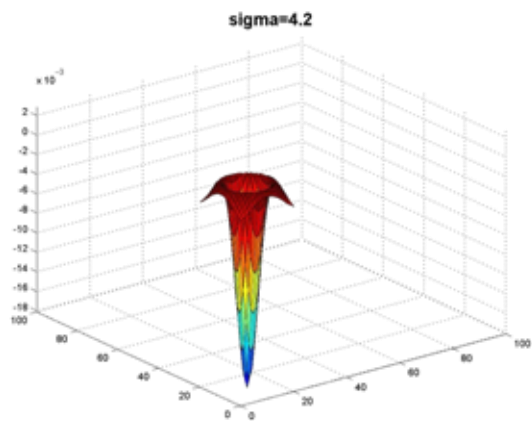
Scaled image
at $\frac{3}{4}$ the size

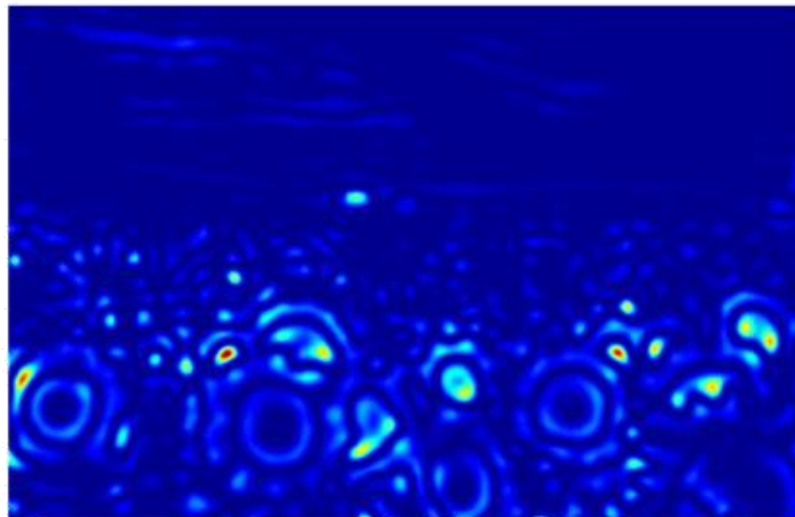
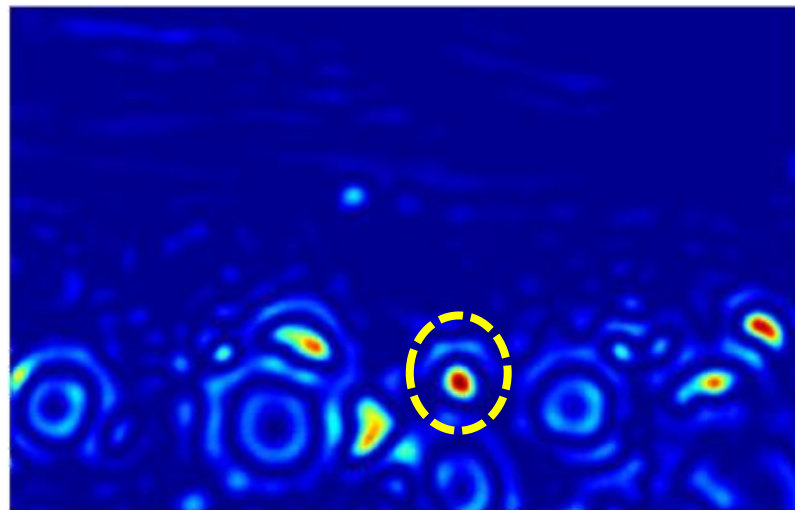
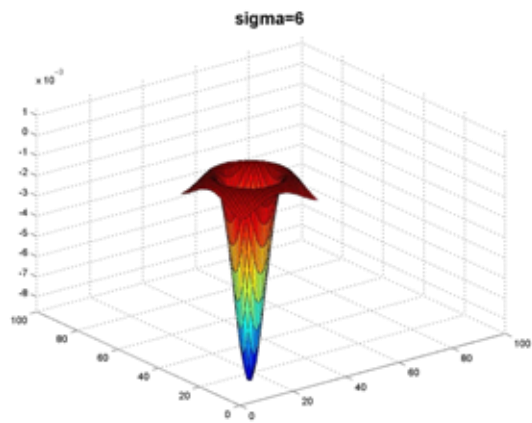


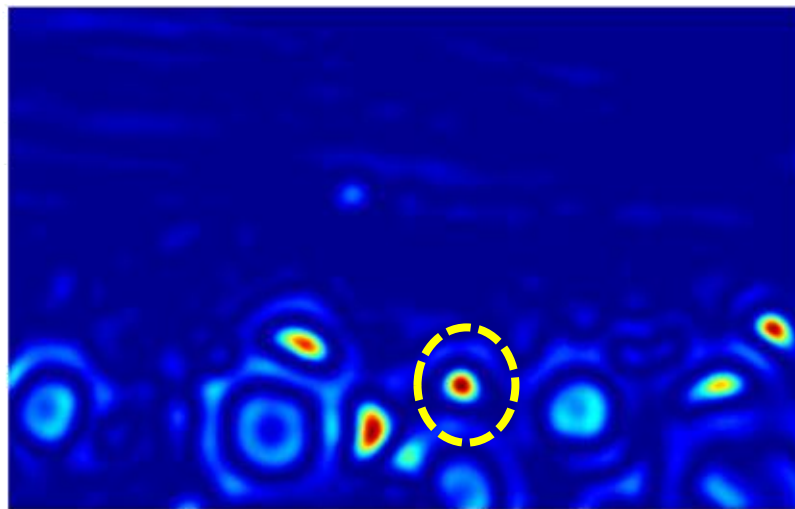
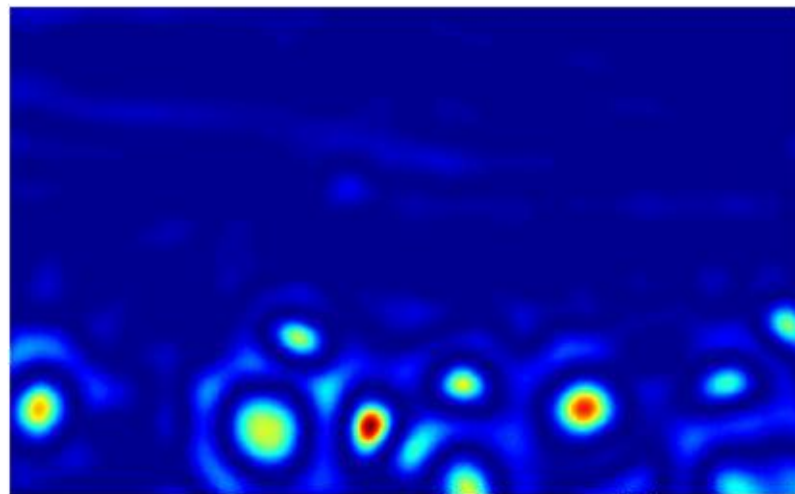
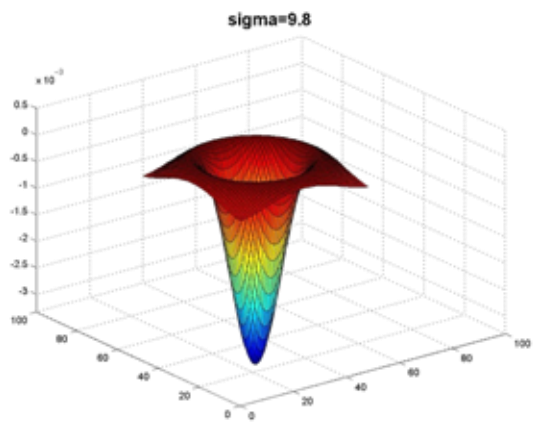
Original image

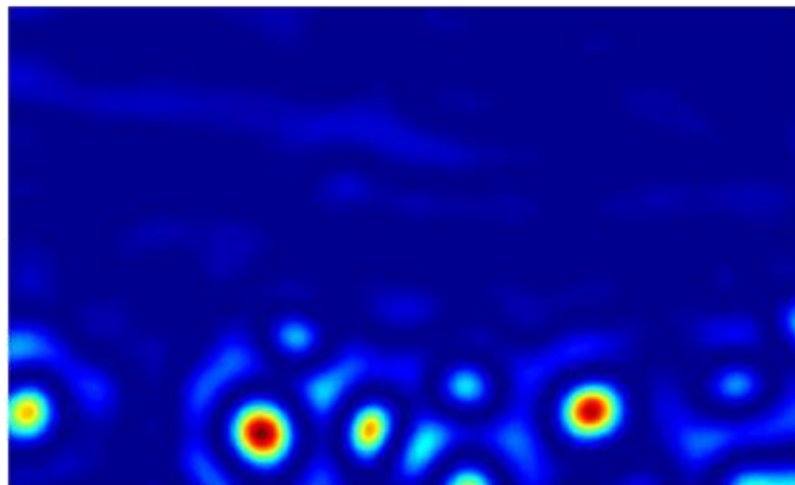
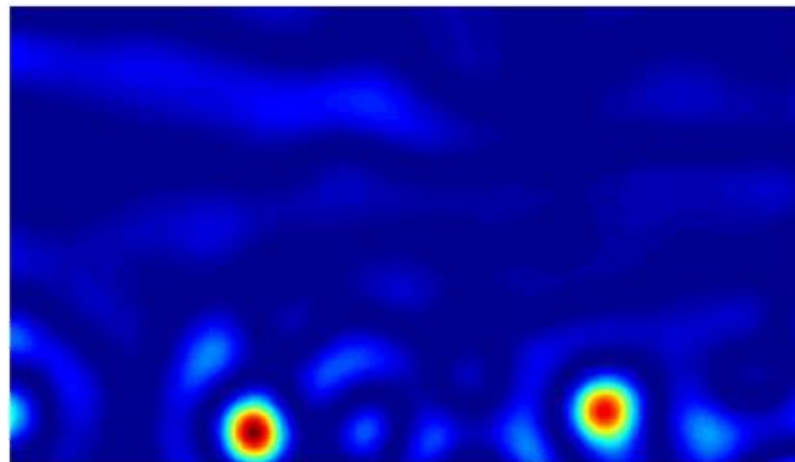
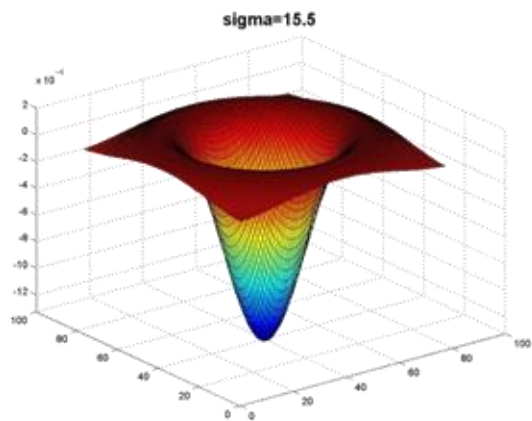


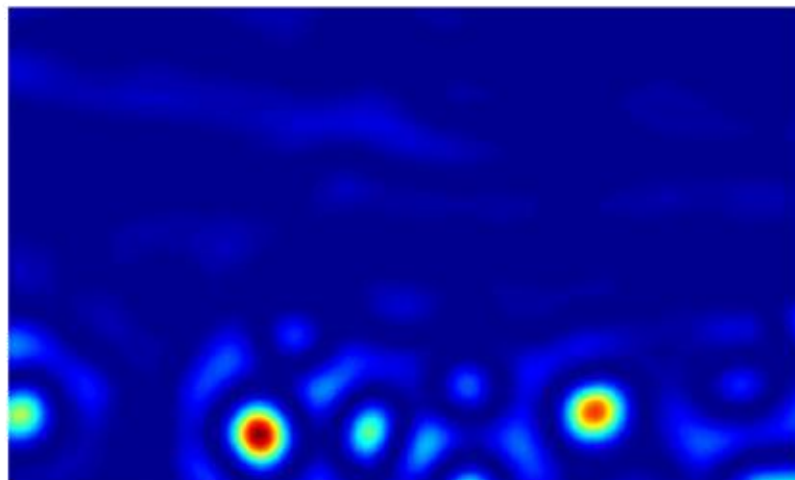
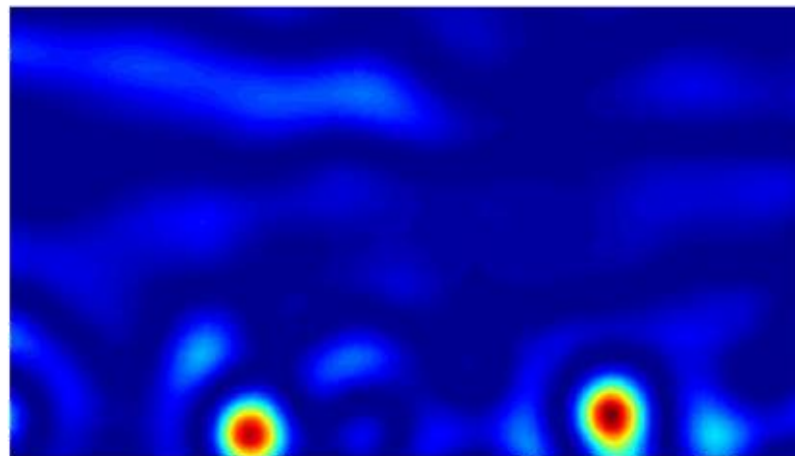
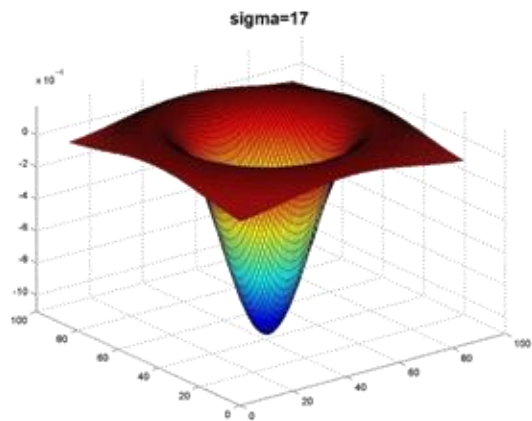












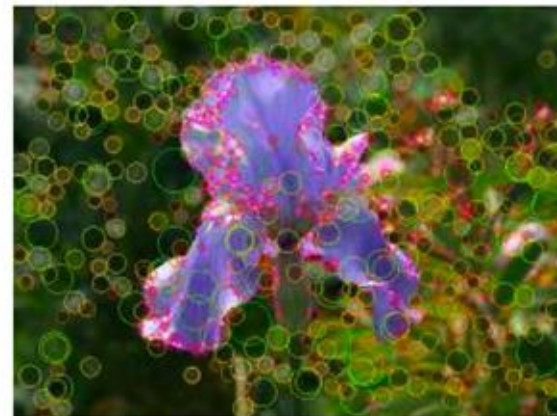
The role of σ

Source: Szeliski



One σ value.

Blobs of the same size detected.

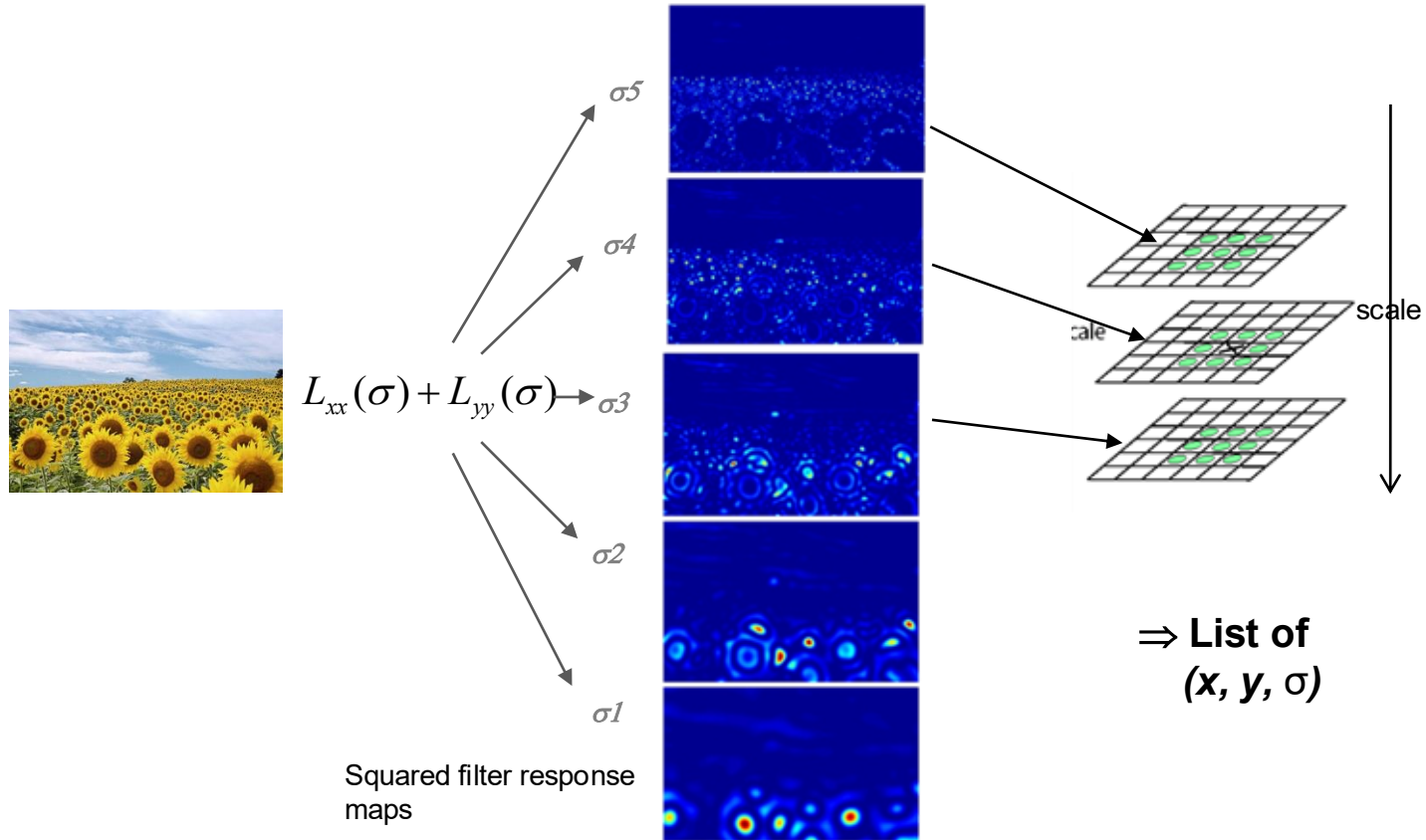


Several σ values.

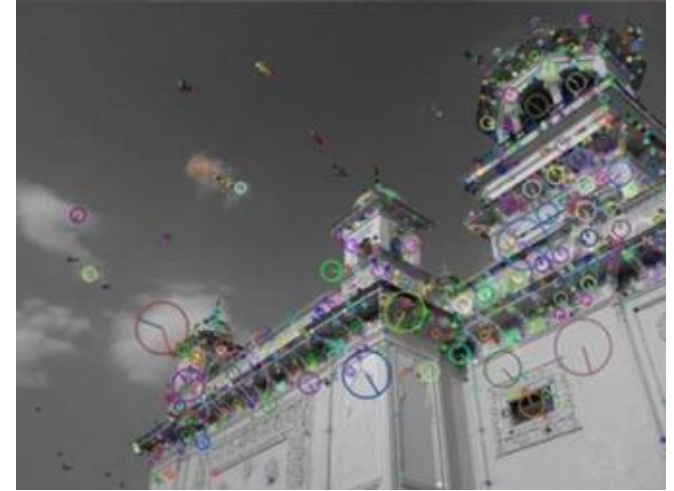
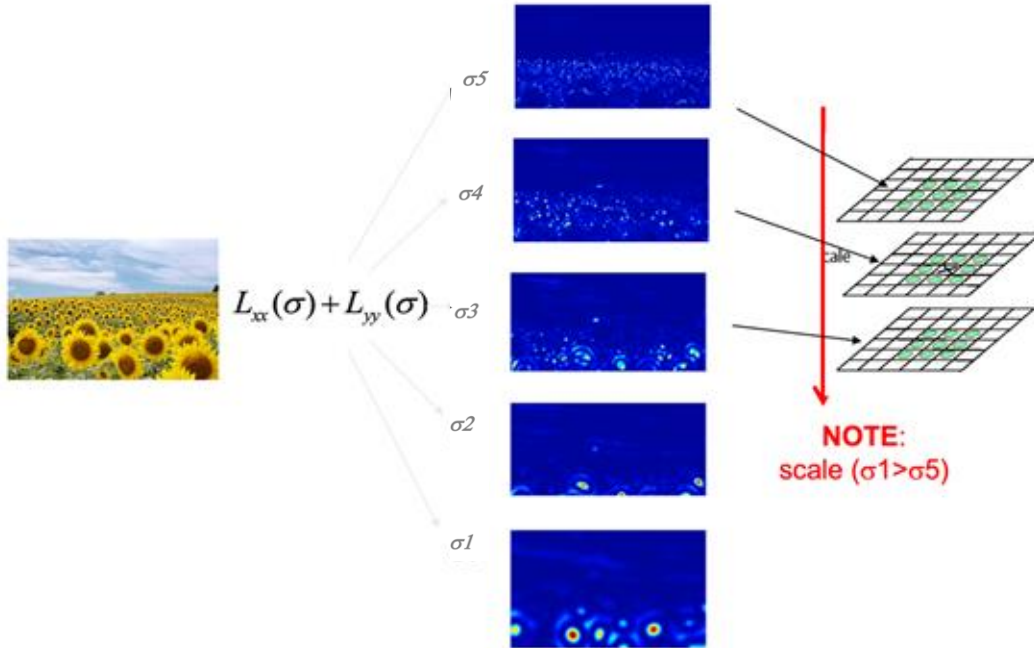
Blobs of various sizes detected.

Scale invariant detection

Interest points are local maxima in both position and scale



Scale invariant detection



[Source: OpenCV SIFT intro](#)

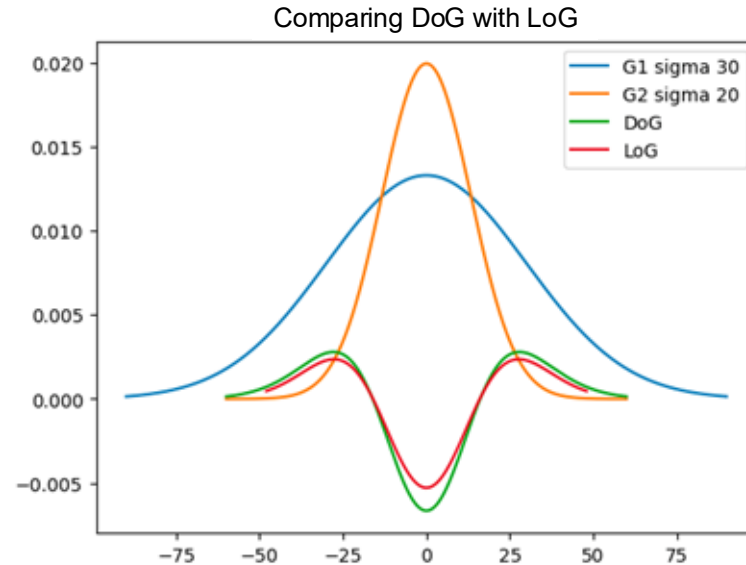
Scale represented by the size of the circle.
Orientation associated to each point, is
the radius defined in the circle.

Scale invariant detection

$$\nabla^2 G_\sigma \approx G_{\sigma_1} - G_{\sigma_2}$$

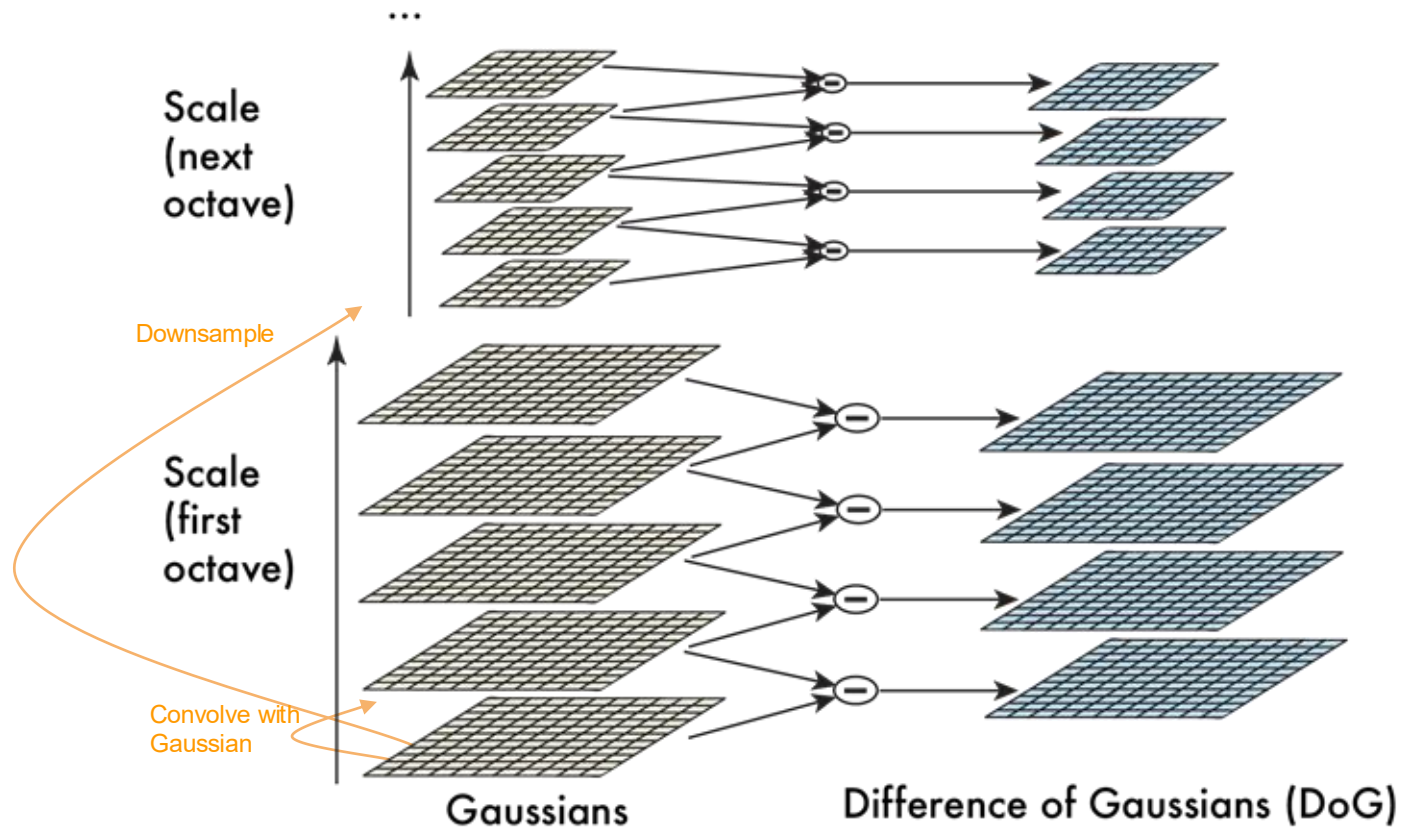
Best approximation when: $\sigma_1 = \frac{\sigma}{\sqrt{2}}, \sigma_2 = \sqrt{2}\sigma$

DoG is similar to the LoG operator, but more efficient



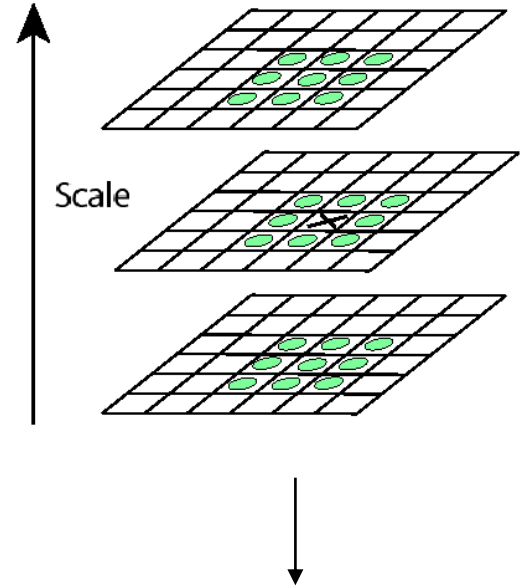
Extract DoG features at multiple scales

LoG can be approximated by a Difference of two Gaussians (DoG) at different scales



Scale invariant detection

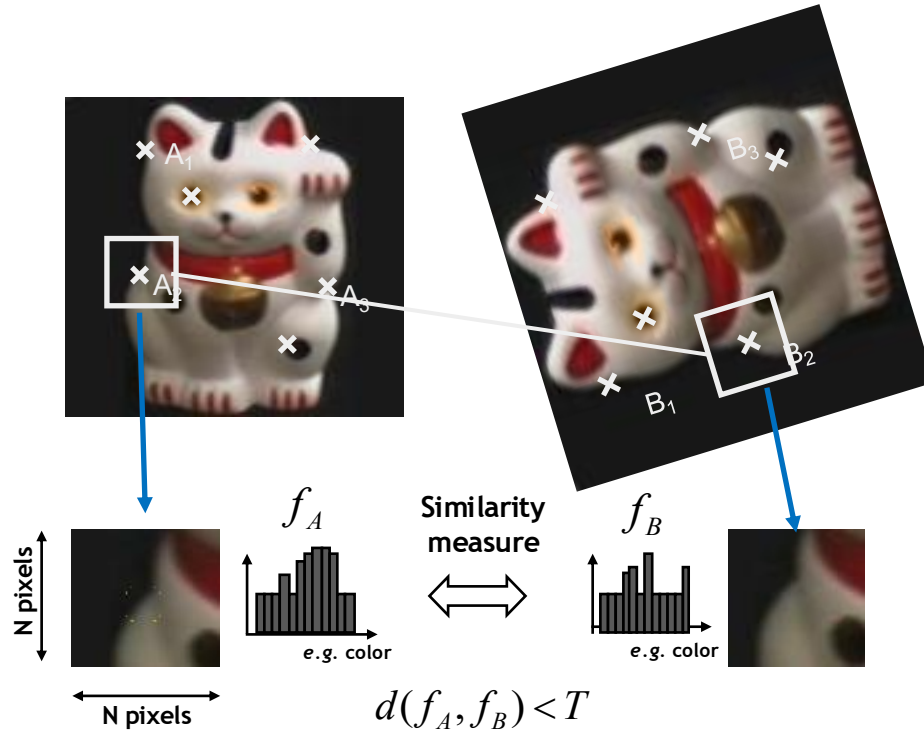
- Detect maxima of DoG in the scale-space volume
- Reject points with weak responses
- Reject points that are localized along an edge



Candidate keypoints:
list of (x, y, σ)

So, what can we do with this?

General Approach

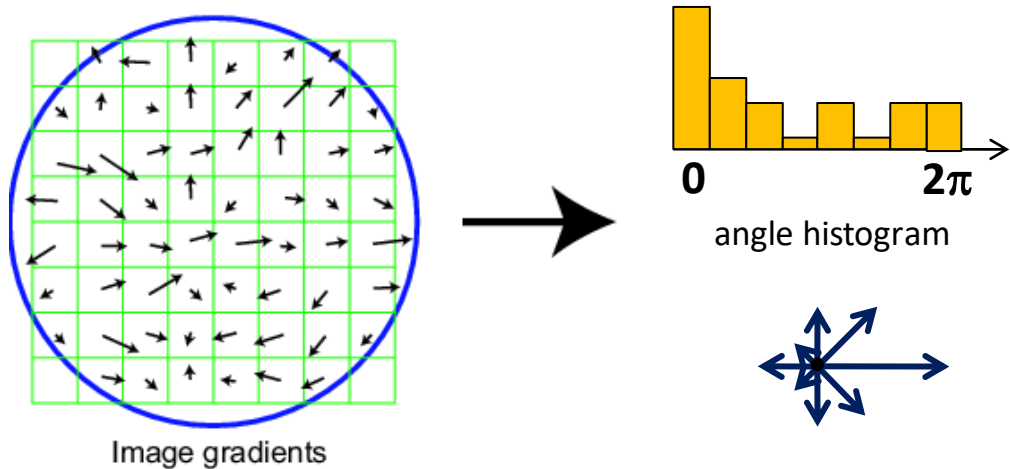


1. Find a set of distinctive key-points
2. Define a region around each keypoint
3. Extract and normalize the region content
4. Compute a local descriptor from the normalized region
5. Match local descriptors

SIFT descriptor

Basic idea:

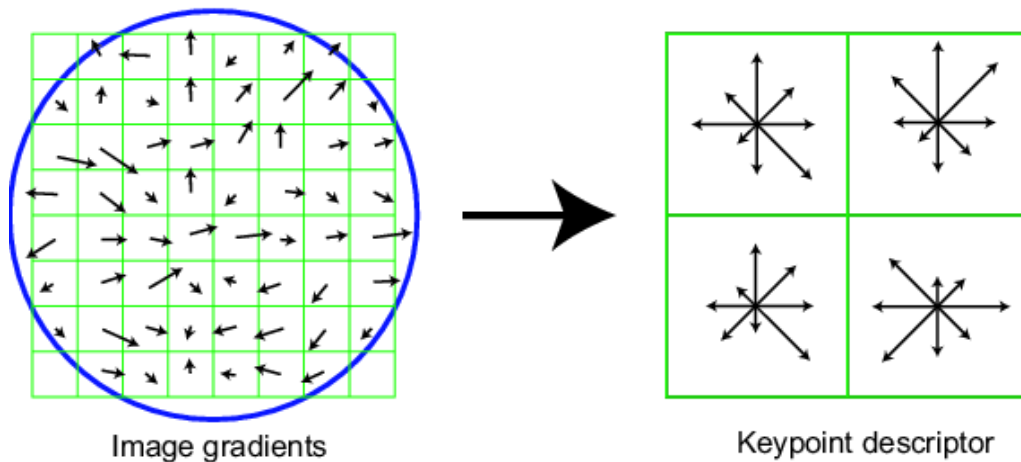
- Take 16x16 square window around detected feature
- Compute edge orientation (angle of the gradient - 90°) for each pixel
- Throw out weak edges (threshold gradient magnitude)
- Create histogram of surviving edge orientations



SIFT descriptor

Full version

- Divide the 16x16 window into a 4x4 grid of cells (2x2 case shown below)
- Compute an orientation histogram for each cell
- 16 cells * 8 orientations = 128 dimensional descriptor

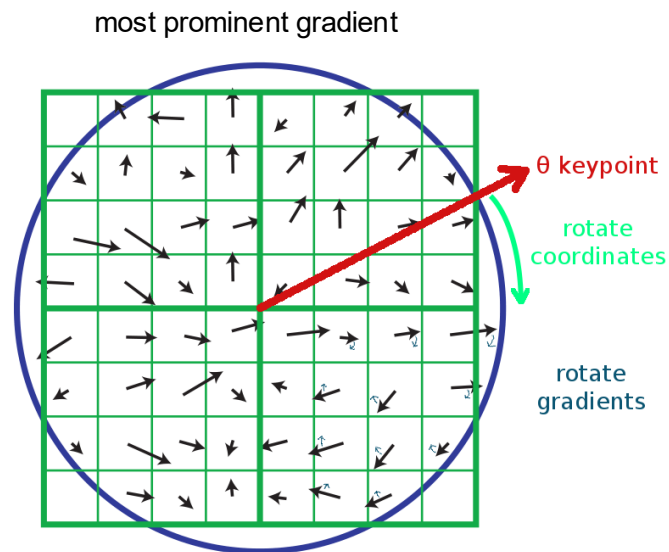


SIFT descriptor

- Descriptors are normalized based on main orientation
- Other normalization steps

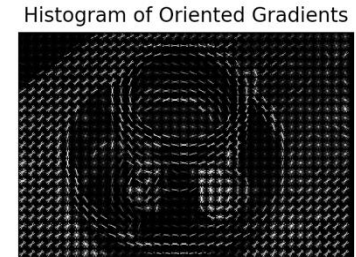
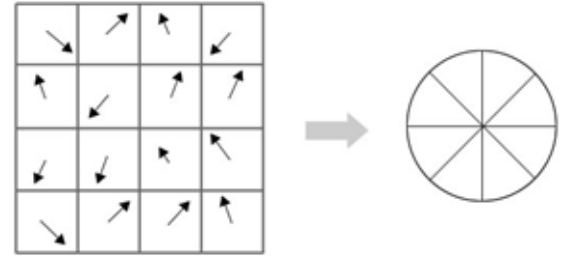
These histograms are invariant to:

- Rotation, translation
- Scale
- Light (we don't use the magnitude)



Histogram of Oriented Gradients (HOG)

- Local Descriptor
- HOG (Dalal and Triggs 2005) consists of:
 - Computing gradients
 - Binning gradients
 - Aggregating blocks (4x4, 16x16 cells)
 - Normalizing gradient magnitudes
- Not reliant on magnitude, just direction
 - Invariant to some lighting changes
 - Not invariant to scale



Object recognition using SIFT



Note: objects can be recognized even when partially occluded or in the presence of clutter



But how can we **match** interest points?

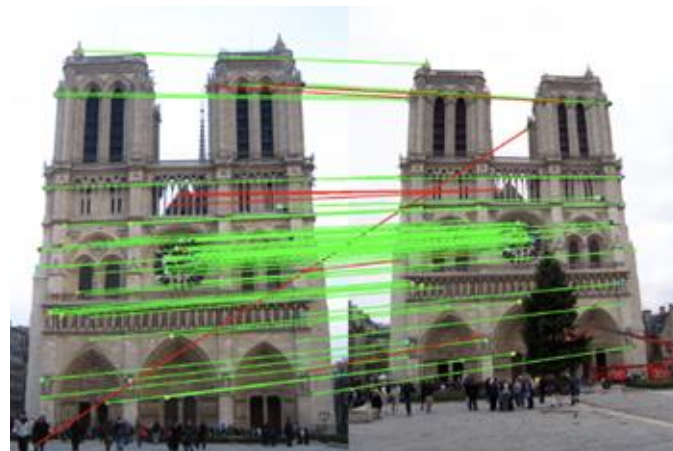
Feature Matching

Find the **correspondences** between the keypoints found in images of the same scene/object.

This match is done using the descriptors of the keypoints from each image.

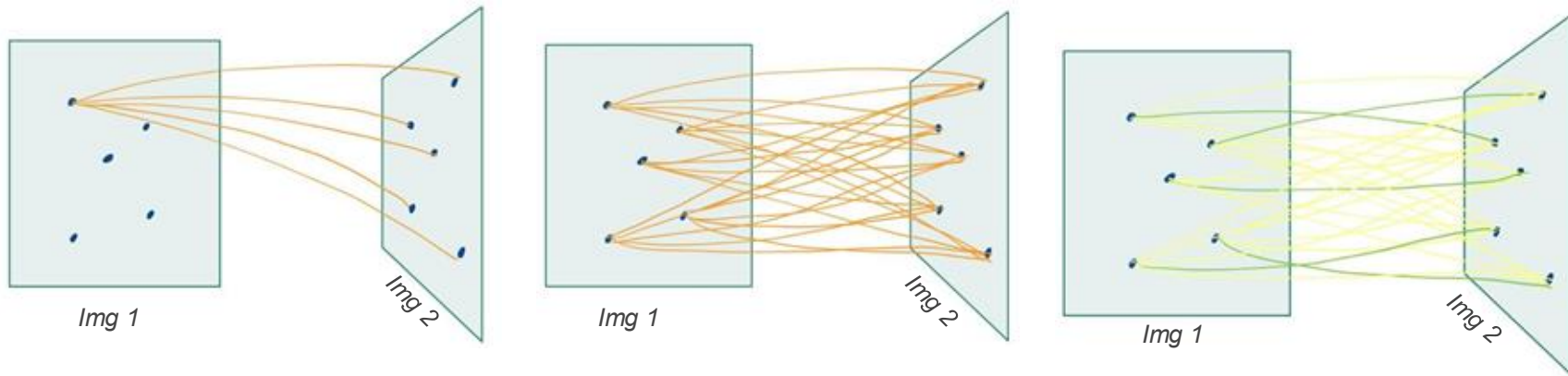
Possible algorithms:

- Brute-force matcher
- FLANN-based matcher (k-d tree)



Brute Force

Takes the descriptor of one feature in first set and is matched with all other features in second set. Compute the distance between each pair.
Select the pairs with closest distance



Disadvantages:

- Computationally expensive

FLANN (Fast Library for Approximate Nearest Neighbors)

It contains a collection of algorithms optimized for fast nearest neighbour search in large datasets and for high dimensional features.

It is **faster** than brute force.

By default, OpenCV uses k-d trees in FLANN, but other methods are available.

K-D tree is a space-partitioning data structure for organising points in a k-dimensional space (special case of binary partitioning trees)



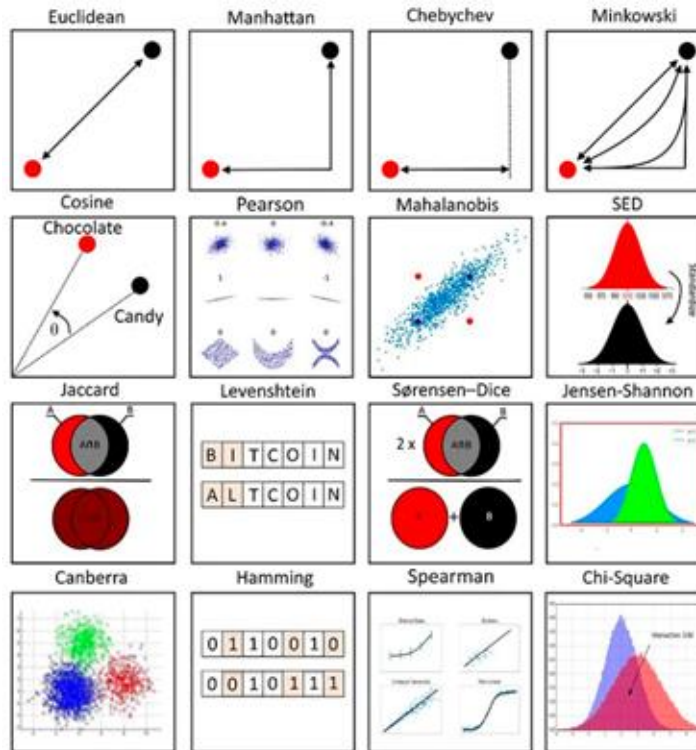
Source: https://docs.opencv.org/3.4/d5/d6f/tutorial_feature_flann_matcher.html

https://docs.opencv.org/4.x/dc/dc3/tutorial_py_matcher.html

Distance functions

Distance between any 2 points
in n-dimensional space.

*The correct choice of distance
measure is crucial for a well
performant model.*



Distance functions

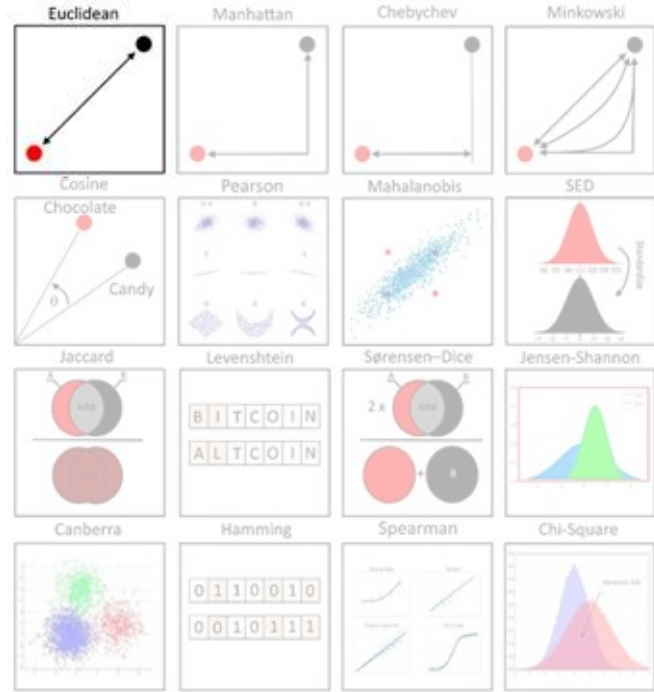
Euclidean / `cv.NORM_L2`

- Most common distance measure
- Length of segment connecting to points, calculated using the Pythagorean theorem

$$D(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

Disadvantages:

- Not scale-invariant
- Impacted by the curse of dimensionality



Distance functions

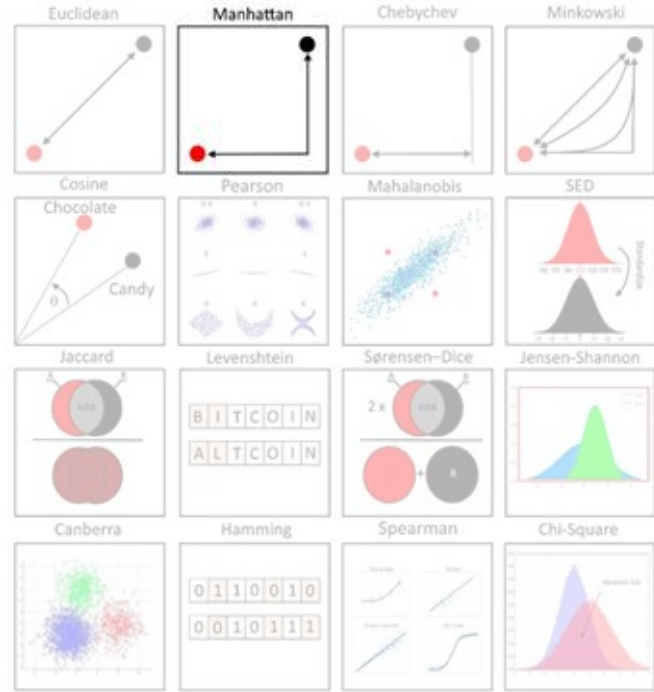
Manhattan / `cv.NORM_L1`

- Also known as taxicab
- It is a simple sum of the magnitudes of two vectors

$$D(x, y) = \sum_{i=1}^k |x_i - y_i|$$

Disadvantages:

- Less intuitive for high-dimensional data



Distance functions

Minkowski

- It is a metric used in normed vector space (n-dimensional real space), it can be used in a space where distances can be represented as a vector that has a length.

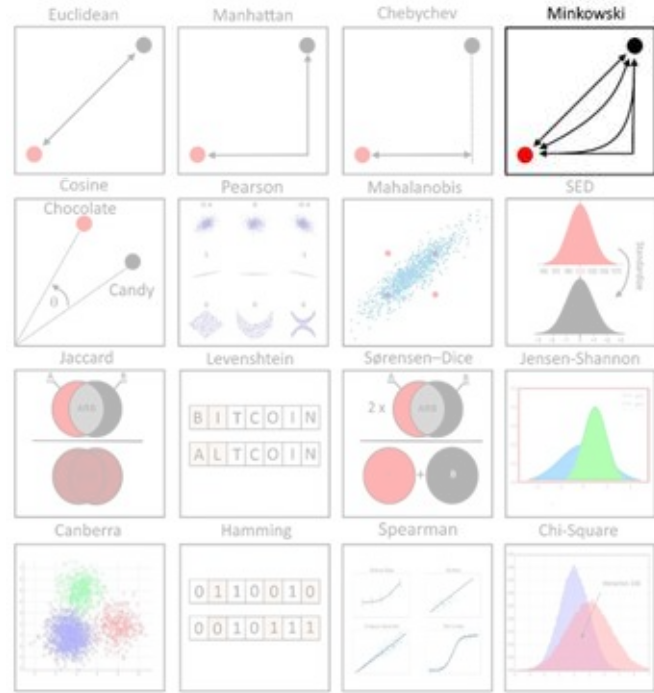
$$D(x, y) = \left(\sum_{i=1}^n |x_i - y_i|^p \right)^{\frac{1}{p}}$$

By changing the parameter **p**, we can convert this measure into other measures, such as:

- $p=1$ — Manhattan distance (city block / taxicab distance)
- $p=2$ — Euclidean distance
- $p=\infty$ — Chebyshev distance

Disadvantages:

- Similar disadvantages as those it represents



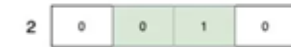
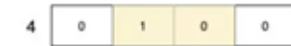
Distance functions

Hamming similarity / `cv.NORM_HAMMING`

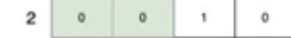
- is the number of values that are different between two vectors of the same size
- useful for binary descriptors



HammingDistance(4,14) = 2



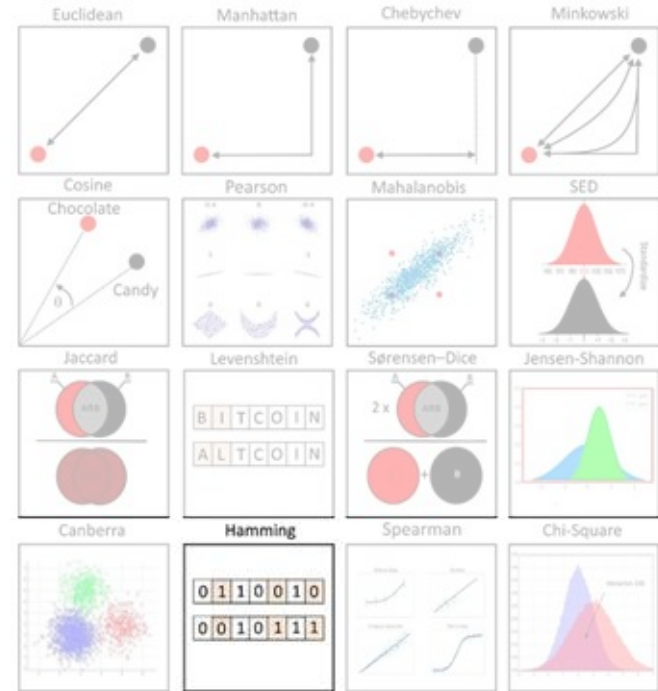
HammingDistance(4,2) = 2



HammingDistance(14,2) = 2

Disadvantages:

- Difficult to use when the vectors don't have the same size



Feature matching

Given a feature in I_1 , how to find the best match in I_2 ?

1. Define **distance function** that compares two descriptors
2. Test all the features in I_2 , find the one with min distance



I_1

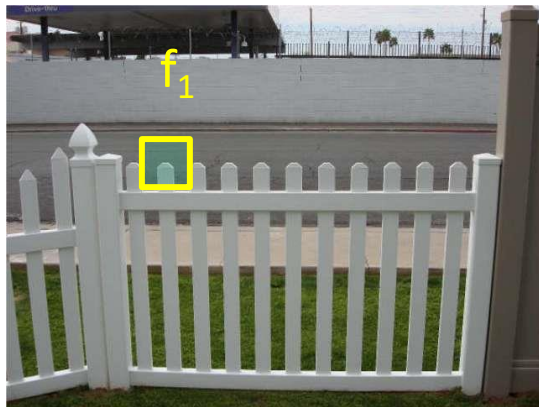


I_2

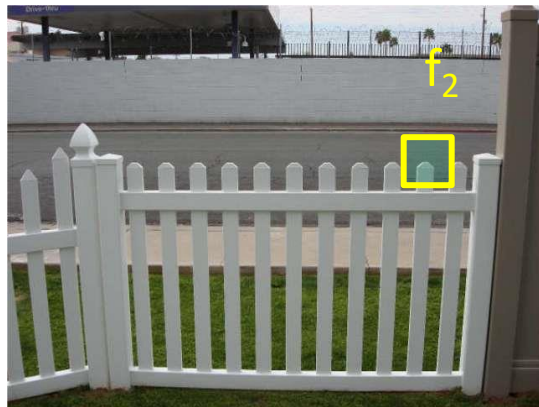
Feature matching

How to define the difference between two features f_1, f_2 ?

- Let us use the simplest approach - $SSD(f_1, f_2)$ or the Euclidean distance
 - sum of square differences between entries of the two descriptors
 - however, it can give good scores to very ambiguous (bad) matches



I_1

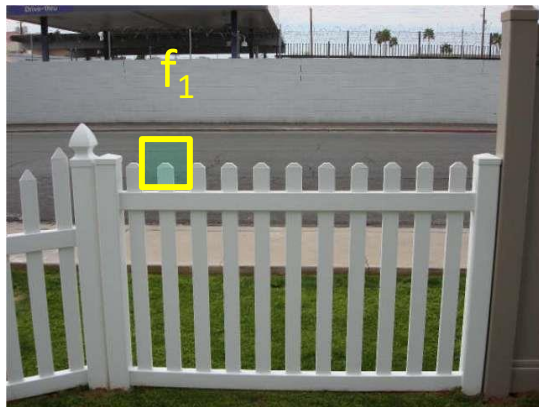


I_2

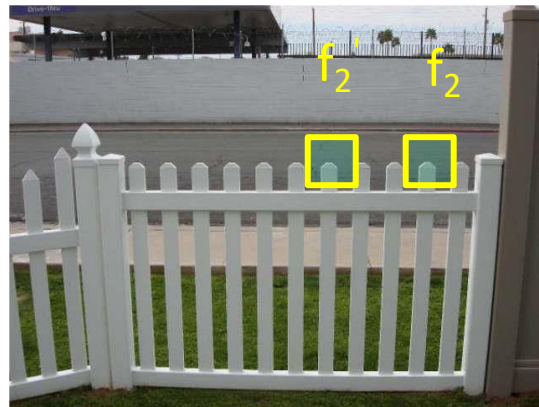
Feature matching

How to define the difference between two features f_1, f_2 ?

- Ratio distance = $SSD(f_1, f_2) / SSD(f_1, f_2')$
 - f_2 is best SSD match to f_1 in I_2
 - f_2' is 2nd best SSD match to f_1 in I_2
 - gives large values (~ 1) for ambiguous matches

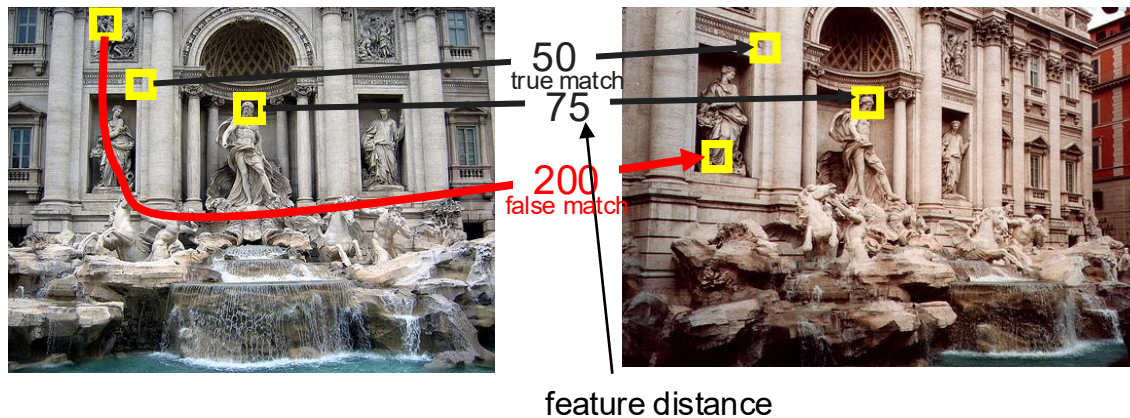


I_1



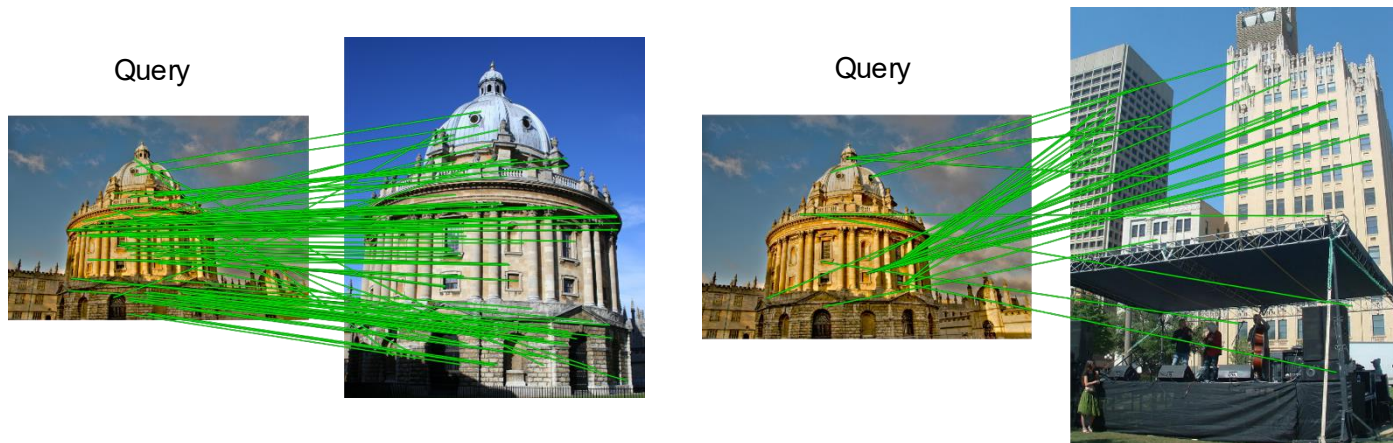
I_2

Feature matching



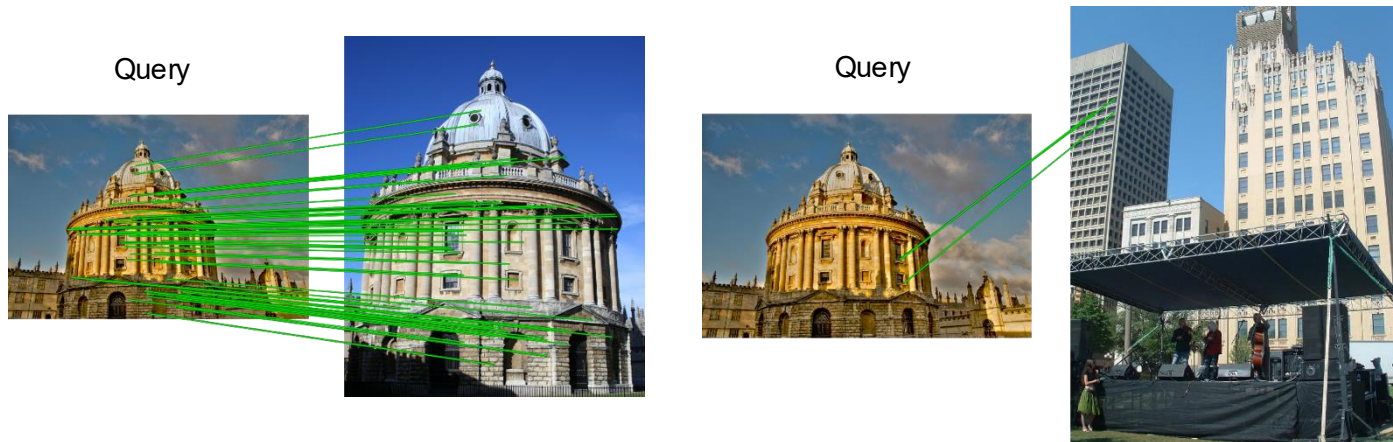
- Eliminate **bad matches**: throw out features with distance $>$ threshold
- And, eliminate matches that are not consistent...

Spatial verification



Both image pairs have many matches.

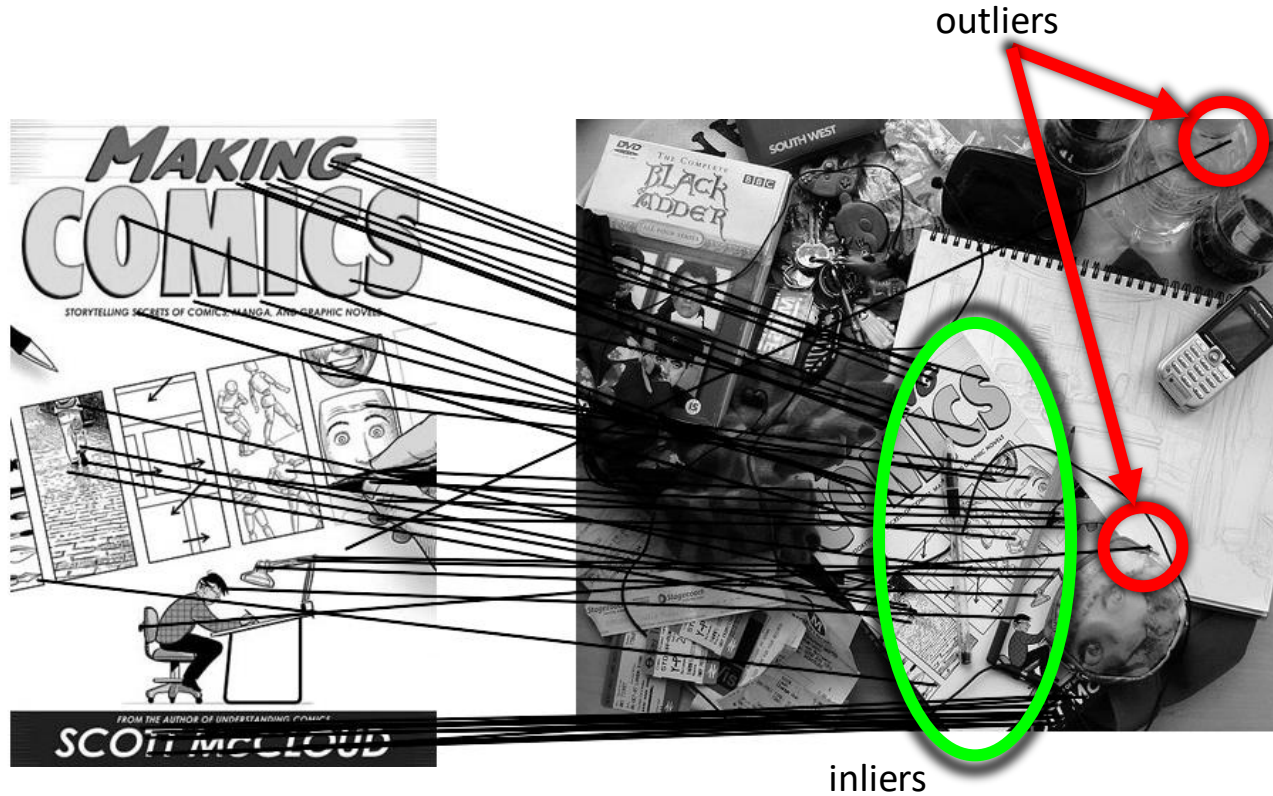
Spatial verification



Only some of the matches are mutually consistent

We need to exclude all other matches (i.e. outliers)

Outliers



We are going to use RANSAC to eliminate these outliers...

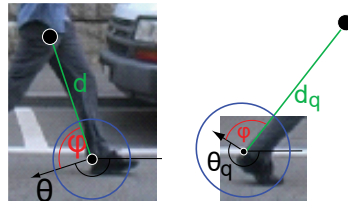
Summary – Local Features

- Global representations have major limitations
- Instead, describe and match only local regions
- Increased robustness to

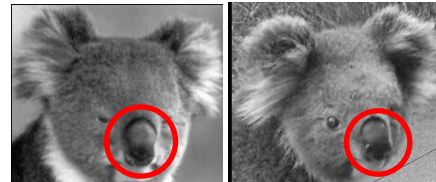
- Occlusions



- Articulation

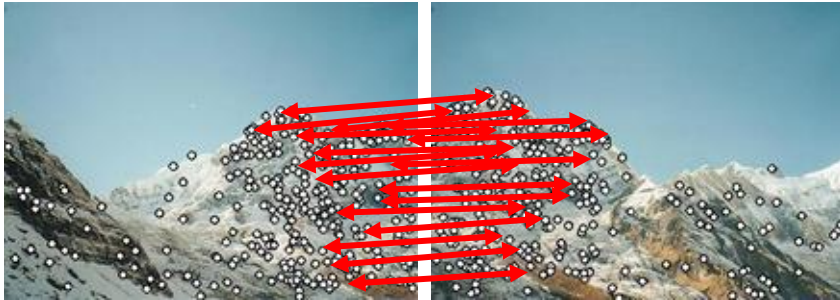


- Intra-category variations



Summary – Local Features

- Highly descriptive local regions
- Ways to describe those regions
- Useful for:
 - Matching
 - Recognition
 - Detection



Open CV

Feature Detection, Description and Matching

Feature Detectors:

- SIFT
- SURF
- MSER
- FAST
- BRISK
- ORB
- AKAZE
- KAZE
- STAR
- MSD

Feature Descriptors:

- SIFT
- SURF
- BRIEF
- BRISK
- ORB
- KAZE
- AKAZE
- FREAK
- DAISY
- LATCH
- LUCID

Feature Matchers:

- Brute-force (BF)
- FLANN

Correspondence rejection:

- Lowe's ratio
- RANSAC

Additional information

Harris Corner detection

1. Maximise the E function (also called Sum Squared Differences (SSD)). Applying the Taylor's expansion we can convert the previous expression to a Matrix form.

$$E(u, v) \approx [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

2. We get the corner matrix (M):

$$M = \sum_{x,y} w(x, y) \begin{bmatrix} I_x I_x & I_x I_y \\ I_x I_y & I_y I_y \end{bmatrix}$$

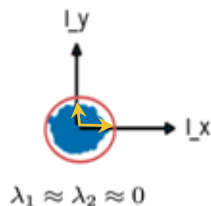
Where I_x and I_y are image derivatives in x and y directions respectively (e.g. using *Sobel*).

3. Then, we calculate the eigenvalues of M and with their magnitude we can classify the region as a flat area, edge or corner.

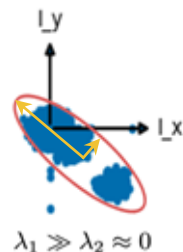
$$R = \det(M) - k(\text{trace}(M))^2$$

- $\det(M) = \lambda_1 \lambda_2$
- $\text{trace}(M) = \lambda_1 + \lambda_2$
- λ_1 and λ_2 are the eigenvalues of M

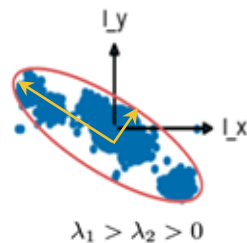
4. For the classification:



If both eigenvalues are small it is a Flat region



If λ_1 is small λ_2 is large it is an Edge region



If both are large than 0 it is a Corner region

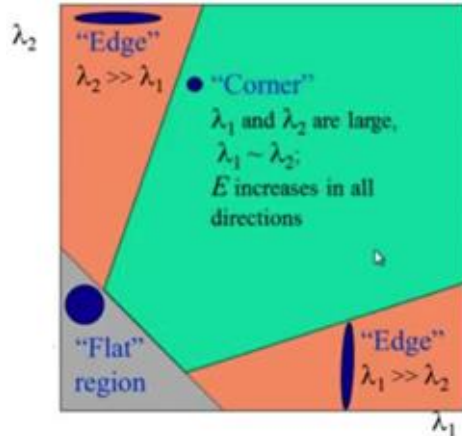
Harris Corner detection

5. The R Harris response calculation can also be determined using the following equation:

The Cornerness function:

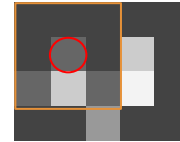
Harris Corner Response Function

$$R = \lambda_1 \lambda_2 - k(\lambda_1 + \lambda_2)^2, k = \text{constant}$$

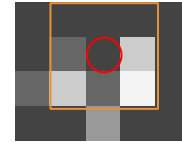


6. Non-Maximal suppression, to remove unwanted detections.

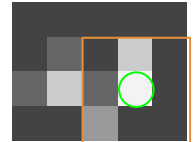
1. Slide a window over the image with the detections
2. At each position, if the central pixel is the maximum value within that window, label it as positive (retain it), otherwise, label it as negative (suppress it)



Suppress



Suppress

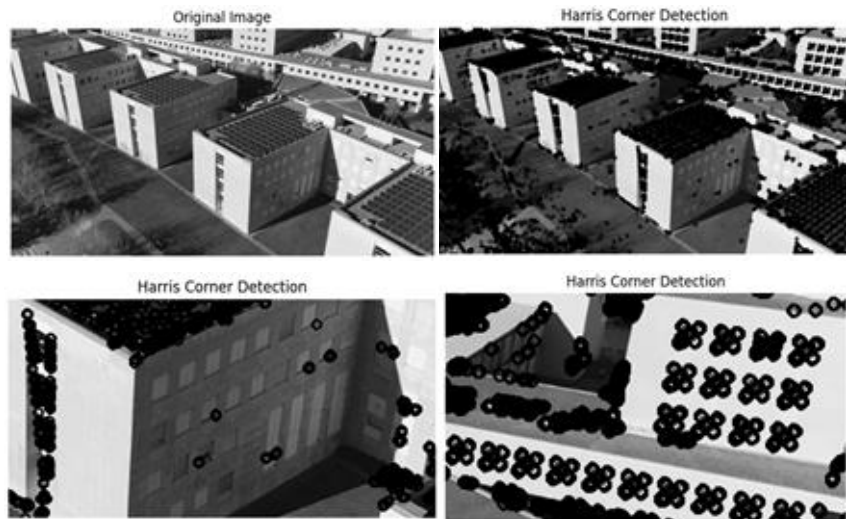


Retain

This non-maximal suppression step helps to select only the most significant corners and discard weaker ones that may be caused by noise or other factors.

Summary | Harris Corner detection | Steps

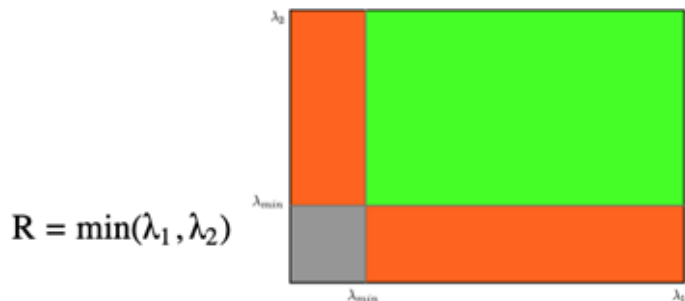
1. Convert color image to grayscale (C)
2. Apply a Gaussian filter to smooth out any noise
3. Slide a window on the image, and compute the spatial derivative, e.g. using Sobel filters
4. Compute the *Sum of Squared Differences* between consecutive windows, convert the expression to a matrix form, find the eigenvalues of the matrix $M (\lambda_1 \lambda_2)$, for every slided window.
5. Generate the Harris response calculation, R , for every window
 - a. find point with maximum score ($R > \text{threshold of your use-case}$)
6. Non-maximum suppression: Take the points in a window and retain local maxima (non-maxima suppression)



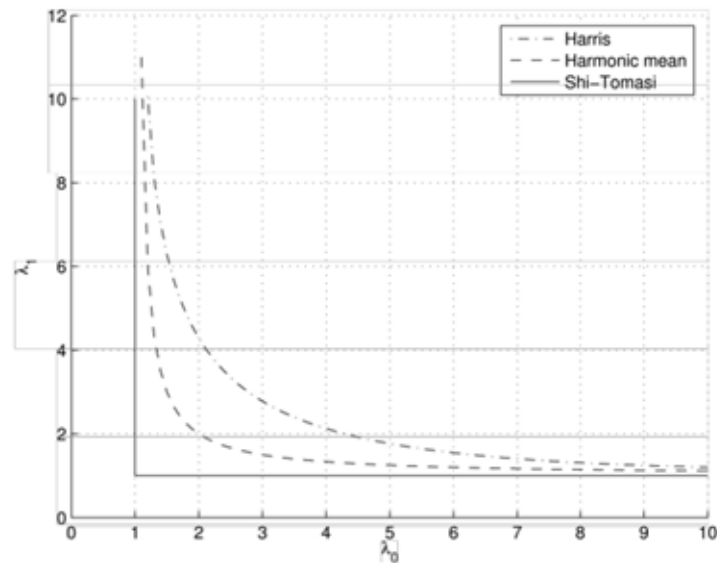
Other variants of Corner detection

- Shi-Tomasi corner detector
https://docs.opencv.org/4.x/d4/d8c/tutorial_py_shi_tomasi.html
- FAST corner detection
https://docs.opencv.org/4.x/df/d0c/tutorial_py_fast.html
- Moravec corner detection (one of the first corner detectors)
<http://rfv.insa-lyon.fr/~jolion/PAPIERS/ptint/node2.html>
- SUSAN (Smallest Univalve Segment Assimilating Nucleus) detector
<http://www.fmrrib.ox.ac.uk/~steve/susan/>

Shi-Tomasi Corner detection is similar to Harris detector, with the exception of R and can find the strongest corners.



Source: [OpenCV Shi-Tomasi Corner Detection](#)



SIFT - Scale Invariant Feature Transform

[D. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints", IJCV, 2004](#)

Provides repeatable and robust features for:

- Recognition
- Panorama stitching
- Tracking

Characteristics:

- Invariant to scaling and rotation
- Partially invariant to illumination and viewpoint changes

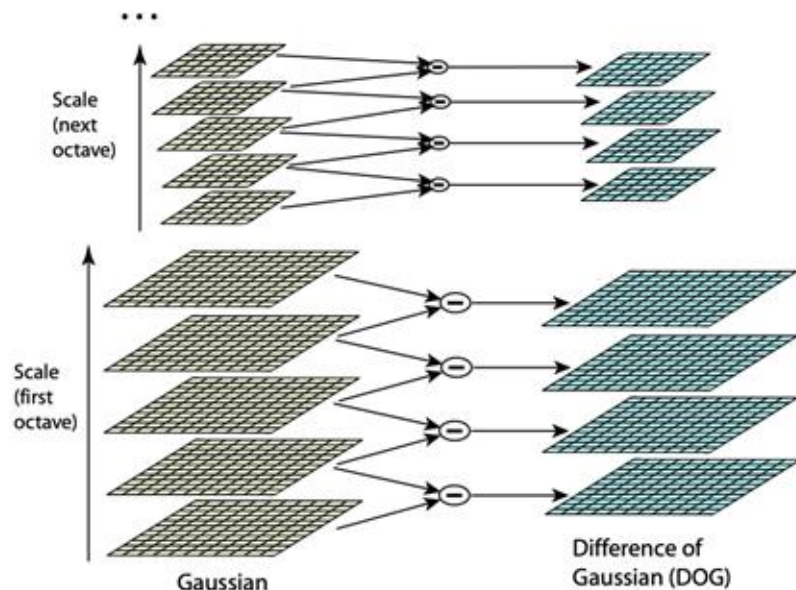


[Source: OpenCV SIFT intro](#)

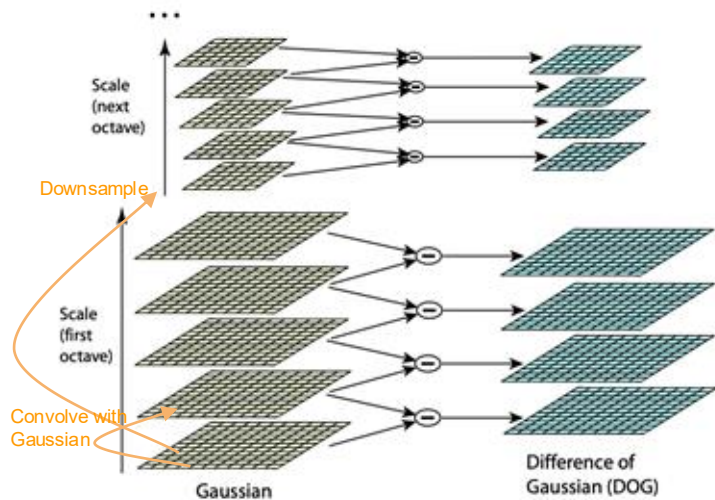
SIFT | Feature Detection Steps

Create scale-space of images:

1. Create different octaves of the input image - same image at $\frac{1}{2}$ resolution
2. For a sequence of consecutive increased σ , filter each octave with each Gaussian
3. Take the differences between the filtered gaussians

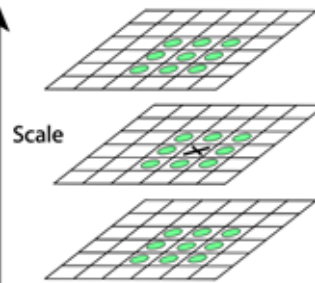


SIFT | Feature Detection Steps



Scale-space feature detection using a sub-octave Difference of Gaussian pyramid.

Adjacent levels of a sub-octave Gaussian pyramid are subtracted to produce Difference of Gaussian images.



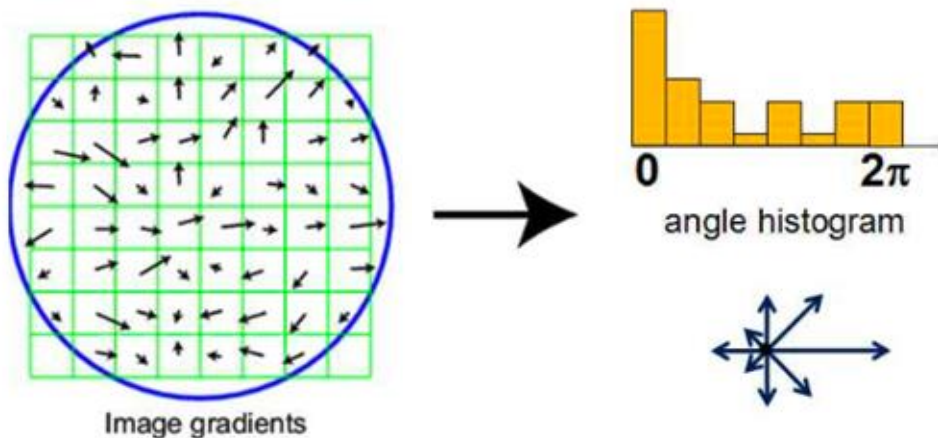
Find extrema (maxima and minima) in the resulting 3D volume are detected by comparing a pixel of its 26 neighbours (3x3x3 grid)

SIFT | Feature Descriptor Steps

Solution: histograms of canonical orientations

We need to build an identifier for each keypoint.

1. Take a 16×16 window around each keypoint
2. Compute the edge orientation of each pixel
3. Throw out weak edges (thresholding gradient magnitude, removing edges)



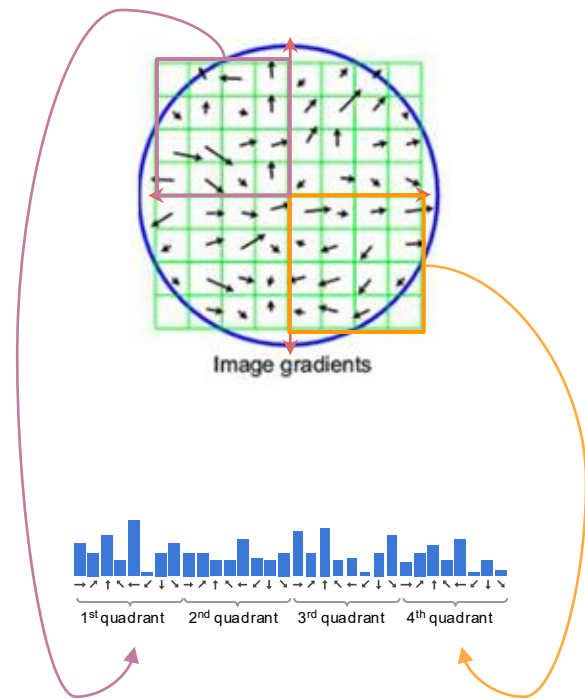
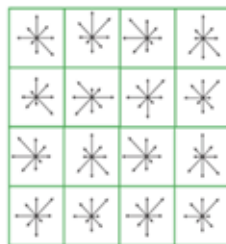
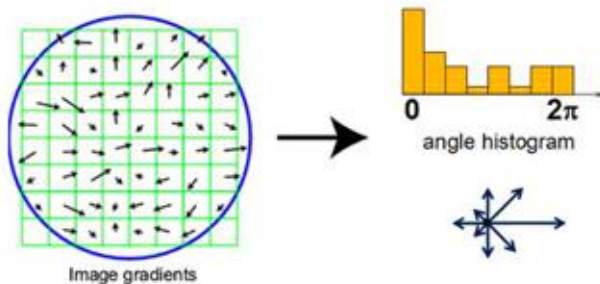
SIFT | Feature Descriptor Steps

Solution: histograms of canonical orientations

We need to build an identifier for each keypoint.

4. Divide the 16x16 window into 4x4 grid of cells
5. Create histograms of surviving edge orientations
6. Concatenate the histograms

→ 16 cells * 8 orientations = 128 dimensional descriptor



Gradient orientation histograms: all the possible directions & the number of pixels in each direction.
 $4 \times 4 \times 8 = 128$ size vector

SIFT | Feature Descriptor Steps

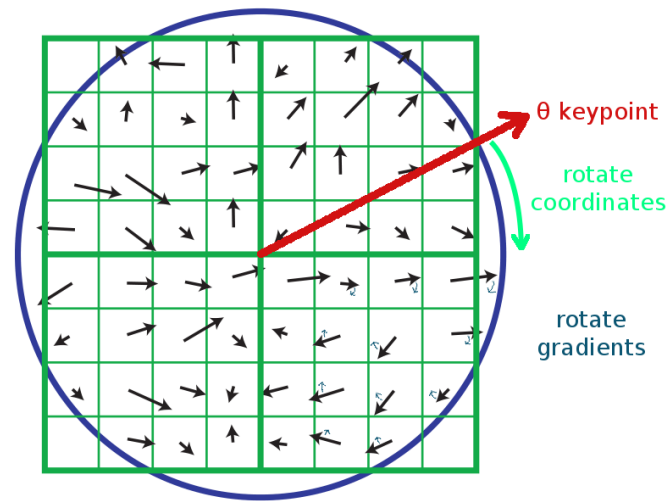
7. Find the orientation with max frequency - main orientation of the keypoint

Orientation adjustment:

- To become rotation invariant (/covariant), SIFT rotates the gradient directions and locations by $(-\theta)$
- θ = keypoint orientation = direction of the max gradient
- Thus, rotation is cancelled out and gradients are expressed at locations relative to keypoint orientation

Contrast normalisation:

- To obtain contrast invariance, SIFT descriptor is normalised to unit sum.
- Thus, the weighted entries in the histogram will be invariant under local affine transformations of the image intensities around the interest point, which improves the robustness of the image descriptor under illumination variations



To handle situations where we might have more than one dominant orientation around the interest point, multiple maximums are accepted if the height of secondary peaks is $> 80\%$ of the height of the highest peak.

Each peak is used for computing a new image descriptor for the corresponding orientation estimate.

Other feature point detectors and descriptors

Detectors:

- Harris https://docs.opencv.org/4.x/dc/d0d/tutorial_py_features_harris.html
- Shi-Tomasi https://docs.opencv.org/4.x/d4/d8c/tutorial_py_shi_tomasi.html
- FAST (Features from Accelerated Segment Test)
https://docs.opencv.org/3.4/df/d0c/tutorial_py_fast.html
- MSER
- CenSurE

Descriptors:

- BRIEF (Binary Robust Independent Elementary Features)
https://docs.opencv.org/4.x/dc/d7d/tutorial_py_brief.html
- Randomized trees
- Ferns
- PCA-SIFT
- GLOH (Gradient Location and Orientation Histogram)
- HOG (Histogram of Oriented Gradients)

Features Detector	Invariance			Qualities			
	Rotation	Scale	Affine	Repeatability	Localization	Robustness	Efficiency
Harris	■	-	-	+++	+++	+++	++
Hessian	■	-	-	++	++	++	+
SUSAN	■	-	-	++	++	++	+++
Harris-Laplace	■	■	-	+++	+++	++	+
Hessian-Laplace	■	■	-	+++	+++	+++	+
DoG	■	■	-	++	++	++	++
Salient Regions	■	■	■	+	+	++	+
SURF	■	■	-	++	+++	++	+++
SIFT	■	■	-	++	+++	+++	++
MSER	■	■	■	+++	+++	++	+++

Source: paper Ehab 2017

Detectors & Descriptors:

- SIFT (Scale-Invariant Feature Transform)
https://docs.opencv.org/4.x/da/df5/tutorial_py_sift_intro.html
- SURF (Speeded-Up Robust Features)
https://docs.opencv.org/4.x/df/dd2/tutorial_py_surf_intro.html
- ORB (Oriented FAST and Rotated BRIEF)
- BRISK (Binary Robust Invariant Scalable Keypoints)
http://docs.opencv.org/modules/features2d/doc/feature_detection_and_description.html

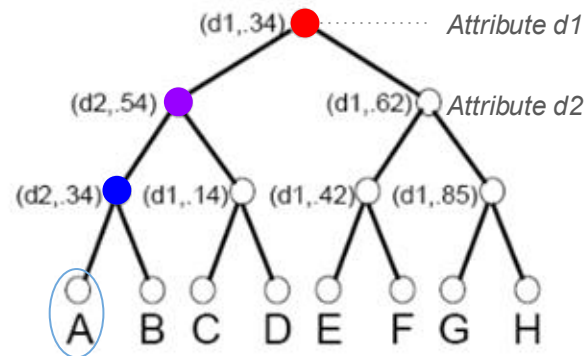
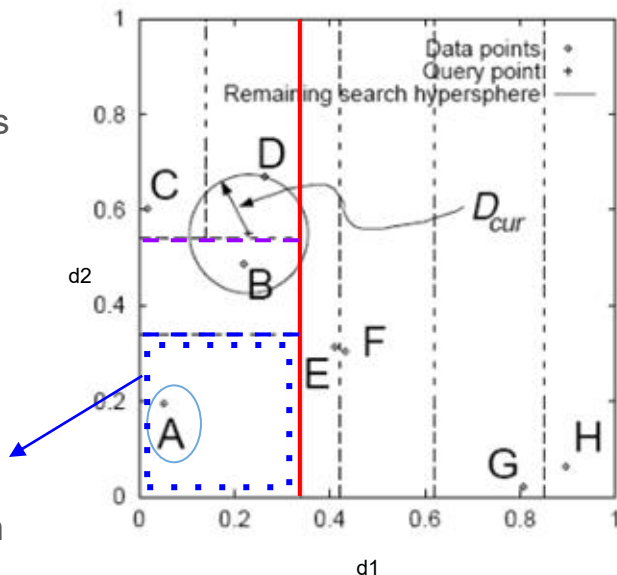
Matching | k-d tree

Building the k-d tree:

- Build with a set of descriptors (“training data”), by picking some dimension (attribute), find its median and split the data.

Matching with k-d tree:

- Given a point compare it with all the points in the region where it is.



Source: Szeliski

Notes:

- Prone to mistakes, because the closest neighbour might be in another region.
- For an N-dimensional space, regions are hypercubes